Created by Dr. Zoltán Subecz, GAMF educational materials were used.
John von Neumann University, Kecskemét, GAMF Faculty of Engineering and Computer Science, Informatics Department
subecz.zoltan@gamf.uni-neumann.hu

Only informative text = we don't study it, only additional information

Web-programming-1 Lecture

In this document, I have written many comments to the code of the exercises. You can find the solutions without explanations in the file **Solutions.zip**.

I have written the explanations after the // signs for simplicity.
These must be removed from the HTML and CSS files for proper operation.
A valid HTML comment would be: <!-- Comment -->

You don't have to memorize the code: there is no Exam in the Lecture, you have to use it only in the Homework.

HTML, CSS, PHP: We deal with these languages and applications a lot in Seminar. We won't review them again here, we'll just use the knowledge we've gained

1-2 – JavaScript - one of the most popular programming languages

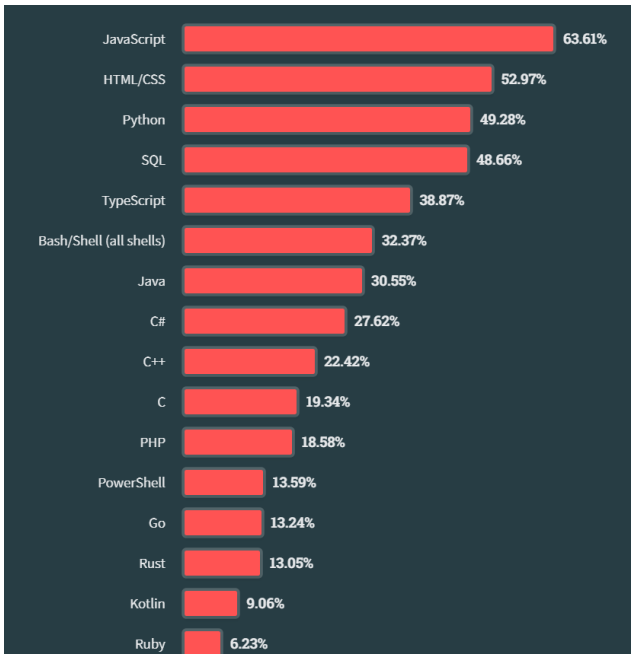
We will also study JavaScript exercises in Seminar

Introduction – Only informative text

Developer: Brendan Eich + Netscape, First standard version: 1997,
Currently used by 98% of websites, NodeJS also uses it

JavaScript is one of the most popular programming languages

https://survey.stackoverflow.co/2023/
How many of those surveyed used it:

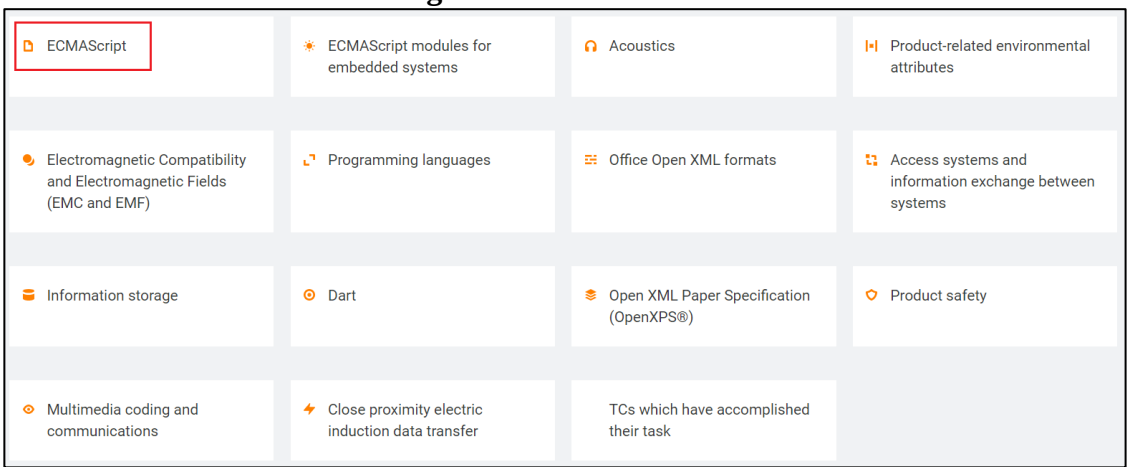


JavaScript is a high-level, often just-in-time compiled language that conforms to the **ECMAScript** standard.

ECMA: Ecma International https://www.ecma-international.org/

Ecma International is an industry association dedicated to the standardization of information and communication systems

Ecma standardization technologies

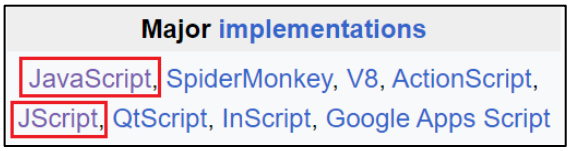


ECMAScript

http://www.ecma-international.org/publications/standards/Stnindex.htm
https://en.wikipedia.org/wiki/ECMAScript

ECMAScript is a JavaScript standard intended to ensure the interoperability of web pages across different browsers.

ECMAScript is commonly used for client-side scripting on the World Wide Web, and it is increasingly being used for writing server-side applications and services using Node.js and other runtime environments.



JScript is Microsoft's legacy dialect of the ECMAScript standard that is used in Microsoft's Internet Explorer 11 and older.
The most popular implementation of ECMAScript is **JavaScript**.

JavaScript was invented by Brendan Eich in 1995, and became an **ECMA standard** in 1997.

ECMAScript Editions

Edition	Date published	Name
1	June 1997	
2	June 1998	
3	December 1999	
4	*Abandoned* (last draft 30 June 2003)	
5	December 2009	
5.1	June 2011	
6	June 2015[10]	ECMAScript 2015 (ES2015)
7	June 2016[11]	ECMAScript 2016 (ES2016)
8	June 2017[12]	ECMAScript 2017 (ES2017)
9	June 2018[13]	ECMAScript 2018 (ES2018)
10	June 2019[14]	ECMAScript 2019 (ES2019)
11	June 2020[15]	ECMAScript 2020 (ES2020)
12	June 2021[16]	ECMAScript 2021 (ES2021)
13	June 2022[17]	ECMAScript 2022 (ES2022)

Hello World! exercise

exercise-01.html

```
<!DOCTYPE html>
<html>
   <head>
      <meta charset="utf-8">
      <title>JavaScript basics</title>
   </head>
   <body>
      <script>
         document.write("Hello World!")
      </script>
   </body>
</html>
```

Printed: Hello World!

Simple event handling - onload Event

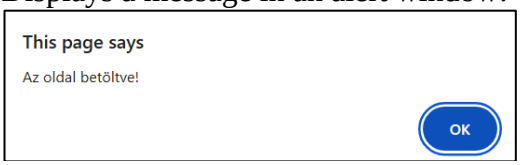
exercise-02.html

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="utf-8">
   <title>JavaScript basics</title>
```

```
    <script>
        function message() {
            alert("The page is loaded!");
        }
    </script>
</head>
<body onload="message()">
    Test page
</body>
</html>
```

onload Event: executes a script once a web page has completely loaded all content.
https://www.w3schools.com/jsref/event_onload.asp

Displays a message in an alert window:



Then after clicking OK it will say: Test page.

If JavaScript is disabled in the browser

If the browser cannot execute the contents of the script element for some reason (e.g. JavaScript is disabled), the contents of the noscript element will be displayed:

```
<script>
document.write("JS is supported.")
</script>
<noscript>
JavaScript is not supported.
</noscript>
```

Variables, If condition, Date

exercise-03.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>JavaScript basics</title>
</head>
<body style="background-color: #eeeeff;">
    <script type="text/javascript">
        var d = new Date()
        var time = d.getHours();
        alert(time);
        if (time < 12)
            document.write("<strong>Good morning!</strong>");
        else
            document.write("<strong>Good afternoon!</strong>");
        var message = "<br />Good " + ((time < 12) ? "morning" : "afternoon") + "!";
```

```
        document.write(message);
    </script>
</body>
</html>
```

in Alert window:

Then prints:
Good morning!
Good morning!

Arrays, For loop

exercise-04.html

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8">
        <title>JavaScript basics</title>
</head>
<body style="background-color: #eeeeff;">
        <script type="text/javascript">
                var mycars = new Array()
                var x
                mycars[0] = "Saab"
                mycars[1] = "Volvo"
                mycars[2] = "BMW"
                for (x in mycars)
                        document.write(mycars[x] + "<br />")
        </script>
</body>
</html>
```

Kiírja:
Saab
Volvo
BMW

Check it:
Right click on page / View page source
/ Inspect

JavaScript is cached by the browser

JavaScript is cached by the browser, so if we change the JavaScript code and refresh the browser, we often do not see the effect of the modification.

Temporarily disable caching in Chrome (while the Inspector is open):

right click on page / Indpect / Network / Disable cache (while DevTools is open)

https://code.mu/en/javascript/book/prime/basis/files-caching
https://marian-caikovski.medium.com/javascript-modules-and-browser-cache-4050b72ec51c

JavaScript Scope, Block, Function(Local) , Global

JavaScript Scope is the area where a variable (or function) exists and is accessible.
Scope determines the usability and visibility of variables. There are 3 types of scope in JavaScript:

- Block block: {…}
- Function(Local) inside function
- Global defined outside functions

var, let, const

Var – not recommended: left over from old JavaScript versions– therefore Only informative text instead: let, const recommended

https://www.w3schools.com/js/js_variables.asp

Using var (Not Recommended)

https://www.freecodecamp.org/news/var-let-and-const-whats-the-difference/

Before the advent of ES6, var declarations ruled. There are issues associated with variables declared with var, though. That is why it was necessary for new ways to declare variables to emerge: **let, const**

https://dev.to/dharamgfx/the-hidden-dangers-of-using-var-in-javascript-why-its-time-to-move-on-2jgm
https://medium.com/swlh/avoid-using-var-in-javascript-422394ed11a3

Variables declared with the var keyword can NOT have block scope.
Var declarations have **global** or **function (local)** scope. The scope is **global** if a var variable is declared outside a function. In this case, the variables can be used throughout the script A var has **function (local)** scope if it is declared inside a function. In this case, it can only be accessed within that function.

no block scope:

```
if(true)
{
  var x = 2;
}
// x CAN be used here: this is one of var problems
document.write(x);     // => 2
// Outputs 2, even though it seems like 'x' should be local to the 'if' block
```

Similar problem:

```
for(var i=1;i<=5;i++)
{
}
// we can use i here as well:
document.write(i);              // => 6
```

There are several problems of var:

https://medium.com/swlh/avoid-using-var-in-javascript-422394ed11a3

local scope – declared is function:

exercise1:

```
function newFunction() {
    var hello = "hello world";
}
newFunction();
document.write(hello);        // error: it doesn't print anything because hello is not defined.
```

global scope - declared out of function:

exercise2:

```
var hello = "hello world-global";
function newFunction() {
    hello = "hello world";
}
newFunction();
document.write(hello);        // prints: hello world
```

exercise3:

```
var hello = "hello world-global";
function newFunction() {
   var hello = "hello world";
}
newFunction();
document.write(hello);        // prints: hello world-global
```

let

let has block scope. block: {}

A variable declared with let can only be used within the given block.

exercise1:

```
if (true) {
   let hello = "say Hello instead";
   document.write(hello);  // "say Hello instead"
}
document.write(hello)      // It doesn't print anything because hello is not defined here.
```

exercise2:

```
let greeting = "say Hi";
if (true) {
   let greeting = "say Hello instead";
   document.write(greeting);      // "say Hello instead"
}
document.write(greeting);         // "say Hi"
```

Const

This is also in block scope, like let scope.

Its value cannot be changed:

```
const greeting = "say Hi";
greeting = "say Hello instead";   // error: Assignment to constant variable
document.write(greeting);         // nem ír ki semmit az előző utasítás hibája miatt
```

Named functions, Anonymous functions and arrow function

Named function

```
function sum1(a,b) {                    // name: sum1
   return a+b;
}
document.write(sum1(5, 3)+" ");         // => 8
```

Anonymous function

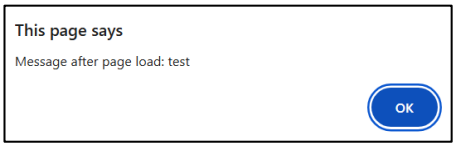
```
   function() {              // it has no name
      // Function Body
    }

   We can call the function e.g. by invoking greet().
   const greet = function () {
      console.log("Hello");
   };
   greet();

var sum2 = function (a,b) {             // sum2: variable
   return a+b;
}
document.write(sum2(5, 3)+" ");         // => 8
```

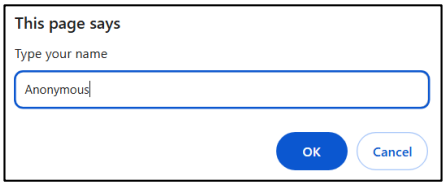
arrow function

```
var sum3 = (a, b) => a+b;
document.write(sum3(5, 3));             // => 8
```

Arrow functions allow you to specify functions more concisely.

Alert box, Prompt box, onclick event





Hello Anonymous!

exercise-05.html

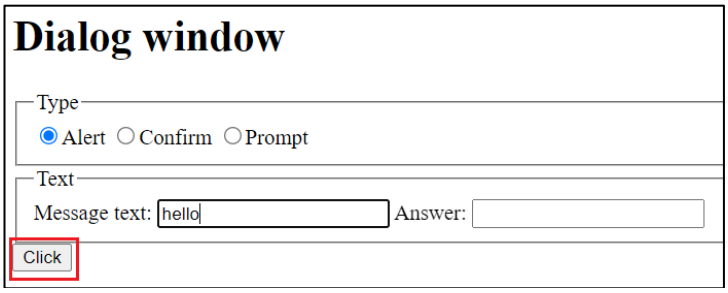
```
<!DOCTYPE html>
<html>
<head>
   <meta charset="utf-8">
   <title>JavaScript basics</title>
   <script type="text/javascript">
      a = "test";
      function message() {
         alert("Message after page load: " + a);
      }
      function display_prompt() {
```

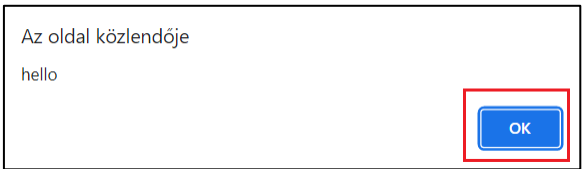
```
        var name = prompt("Type your name","Anonymous")
        if (name != null && name != "")
            document.write("Hello " + name + "!")
    }
  </script>
</head>
<body onload="message()">
  <input type="button" onclick="display_prompt()" value="Button">
</body>
</html>
```

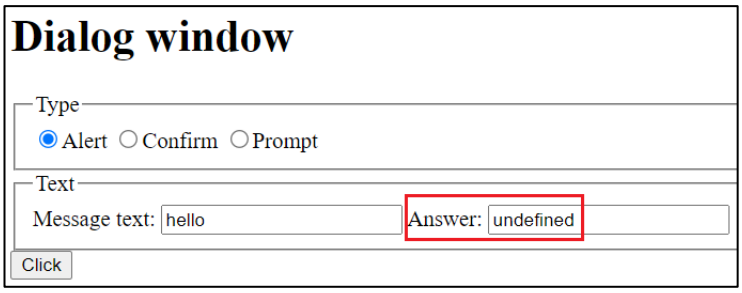
Alert box, Confirm box, Prompt box, switch-case

The Answer text box is read only.

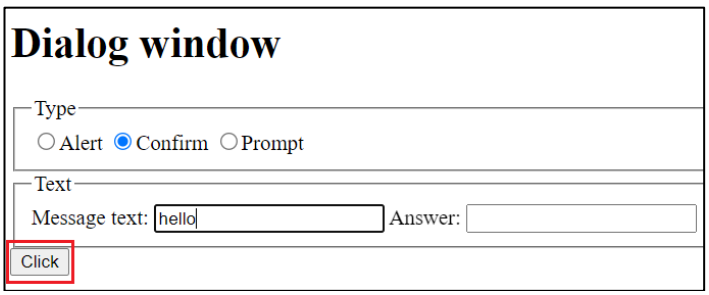
with Alert box

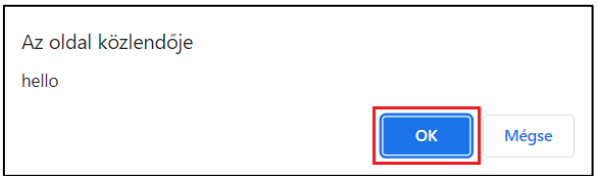


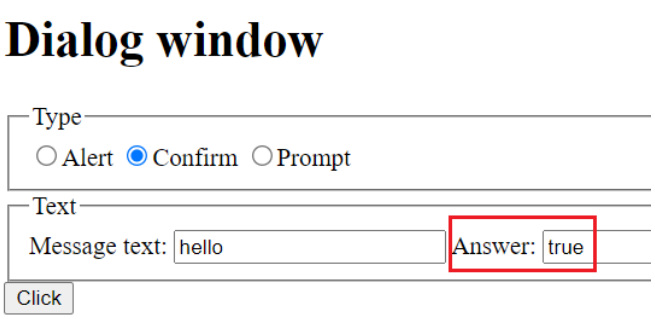




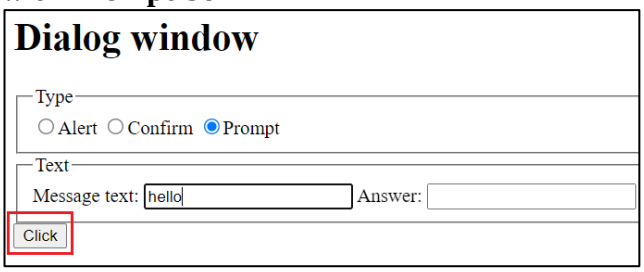
with Confirm box

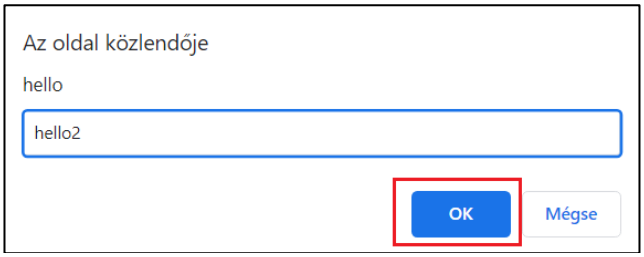


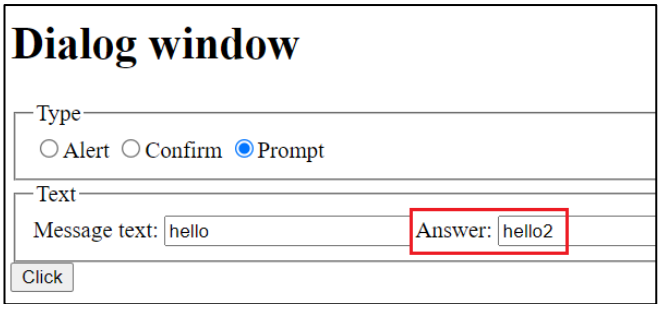




with Prompt box





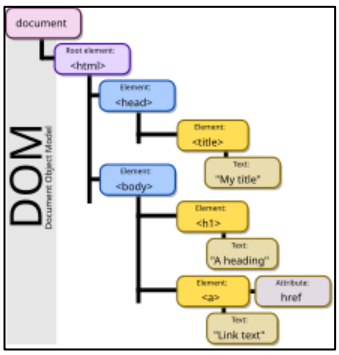


exercise-06.html

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="utf-8">
   <title>JavaScript basics</title>
   <script type="text/javascript">
      function message() {
          switch (true) {
         case document.form1.type1[0].checked:
            document.form1.answer.value = alert(document.form1.text1.value)
            break
         case document.form1.type1[1].checked:
            document.form1.answer.value = confirm(document.form1.text1.value)
            break
         case document.form1.type1[2].checked:
            document.form1.answer.value = prompt(document.form1.text1.value)
            break
          }
      }
   </script>
</head>
<body>
   <h1>Dialog window</h1>
   <form name="form1">
      <fieldset>
         <legend>Type</legend>
         <label><input type="radio" name="type1">Alert</label>
         <label><input checked="checked" type="radio" name="type1">Confirm</label>
         <label><input type="radio" name="type1">Prompt</label>
      </fieldset>
      <fieldset>
         <legend>Text</legend>
```

```
        <label>Message text: <input type="text" name="text1"></label>
        <label>Answer: <input type="text" name="answer" readonly></label>
      </fieldset>
      <input onclick="message();" type="button" name="button" value="Click">
   </form>
</body>
</html>
```

Document Object Model (DOM)



The Document Object Model (DOM) is a cross-platform and language-independent interface that treats an HTML or XML document as a tree structure wherein each node is an object representing a part of the document. The DOM represents a document with a logical tree. Each branch of the tree ends in a node, and each node contains objects.

We can modify/access the page with JavaScript if it is run after the DOM of the HTML document is created!

=> … onload="init()" … It runs when the page loads

Accessing elements of the page e.g.: getElementById(…), getElementsByClassName(…)

getElementById

The **getElementById()** method returns the element with the given ID.

Create the following page:
Open/close the “Work place” box by clicking on the checkbox:

exercise-07.html

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8">
        <title>JavaScript basic</title>
</head>
<body>
   <h1>Dialog window</h1>
```

```
    <input type="checkbox" onclick="document.getElementById('info').style.visibility = this.checked ?
'visible' : 'hidden'"> I work <br>
    <div id="info" style="visibility:hidden">
        Work place: <input name="workplace" type="text">
    </div>
</body>
</html>
```

Clicking a button jumps to a given item

exercise-08.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>JavaScript basics</title>
    <script type="text/javascript">
        function send() {
            var elem = document.getElementById("id1");
            elem.innerHTML="Hello2";
            elem.scrollIntoView();
        }
    </script>
</head>
<body>
    <input type="submit" onclick="send()" value="Write"><br>
    <script type="text/javascript">
        for(var i= 0; i<=100; i++)
            document.writeln("<p>Text</p>");
    </script>
    <H1 id="id1"></H1>
    <script type="text/javascript">
        for(var i= 0; i<=100; i++)
            document.writeln("<p>Text</p>");
    </script>
</body>
</html>
```

Counts button clicks

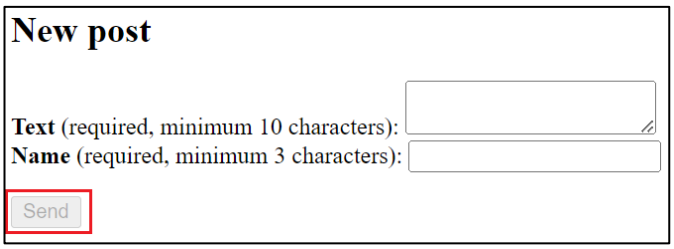
exercise-09.html

```
<html>
<head>
    <title>Button Clicker</title>
</head>
<body>
    <script type="text/javascript">
    var clicks = 0;
    function hello() {
        clicks += 1;
        document.getElementById("clicks").innerHTML = clicks;
```

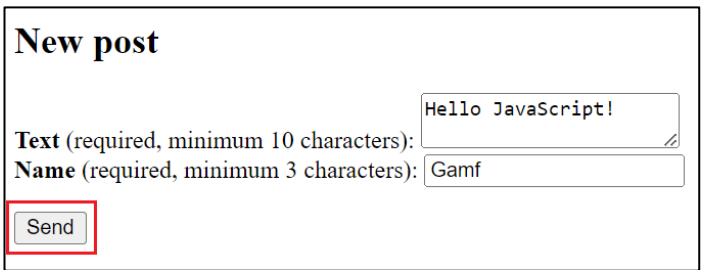
```
    };
    </script>
    <p>Clicks: <a id="clicks">0</a></p>
    <button type="button" onclick="hello()">Click me</button>
</body>
</html>
```

Form - client side validation, addEventListener

The correct completion of the form is first checked on the client side. Only then is it sent to the server side if everything is correct. It is also necessary to check the received data on the server side (server-side validation).
Make the Send button inactive after the page has loaded:



Check each time you type a character to see if the conditions are met. If so, activate the Send button:



The **addEventListener()** method assigns an event handler to an element.

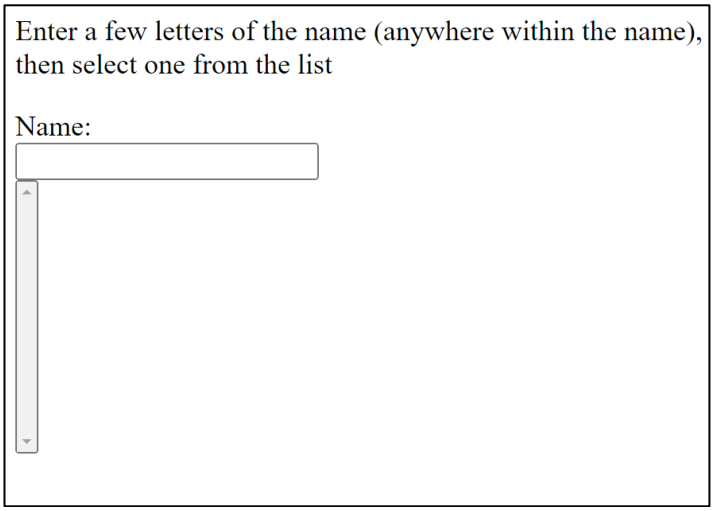
exercise-10.html

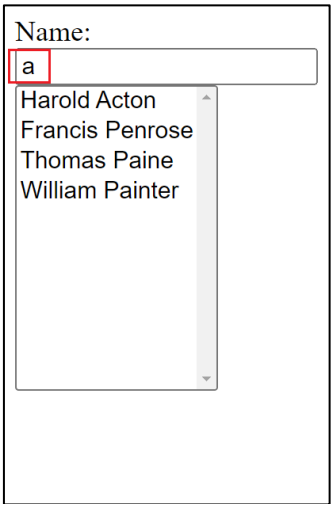
```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>JavaScript basic</title>
    <script type="text/javascript">
        function formCheck() {
            document.getElementById('postsend').disabled = (document.getElementById('text1').value.length
< 10) || (document.getElementById('name1').value.length < 3)
        }

        function init() {
            document.getElementById('postsend').disabled = true;
            document.getElementById('text1').addEventListener('keydown', formCheck);
            document.getElementById('name1').addEventListener('keydown', formCheck);
        }
    </script>
```

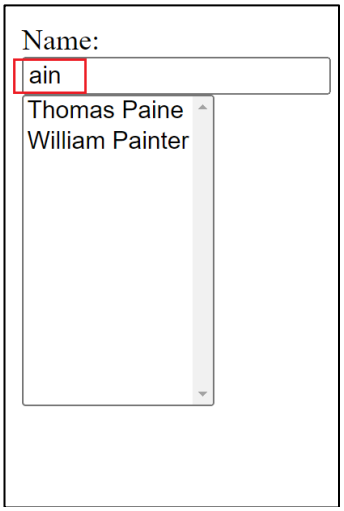
```
</head>
<body onload="init()">
   <h2>New post</h2>
   <label for="text1"><strong>Text</strong> (required, minimum 10 characters):</label>
   <textarea id="text1" name="text1"></textarea><br>
   <label for="name1"><strong>Name</strong> (required, minimum 3 characters):</label>
   <input id="name1" name="name1" type="text" size="20"><br>
   <p></p>
   <button name="postsend" id="postsend" type="submit">Send</button>
</body>
</html>
```

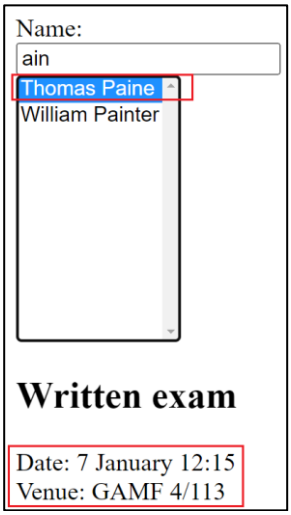
Exercise – Search in a list - nested Array, indexOf

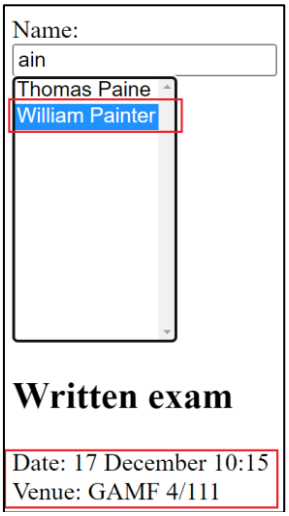
In the program, enter an array of names and the data associated with the names. Enter a few characters of a name (anywhere in the name). Print the names associated with it in a drop-down list. From the printed names, the user chooses one and print the data of the chosen one in a paragraph:











exercise-11.html

```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8">
```

```
	<title>JavaScript basic</title>
	<script type="text/javascript">
		var examinees = new Array(
				new Array("Harold Acton","15 December","8:45","GAMF 4/8"),
				new Array("Henry Green","5 January","13:30","GAMF 4/109"),
				new Array("Stephen Fry","10 January","11:45","GAMF 4/108"),
				new Array("Royston Ellis","12 January","10:45","GAMF 4/110"),
				new Array("Francis Penrose","3 January","12:30","GAMF 4/113"),
				new Array("Thomas Paine","7 January","12:15","GAMF 4/113"),
				new Array("William Painter","17 December","10:15","GAMF 4/111")
			)

		function typing(){
			// Delete all data
			var writtenexam = document.getElementById('writtenexam');
			writtenexam.style.display = 'none';
			// Objects for typing the name and displaying the list of names (name1 and list1)
			var name1 = document.getElementById('name1');
			var list1 = document.getElementById('list1');
			// We build the list in the str variable
			var str = '';
			// If the name field is not empty, a list is generated
			if (name1.value.length>0)
				for (i=0; i<examinees.length; i++) {
					var exname = examinees[i][0];
					// If the current name contains the sample text (name1.value), then a
good name has been found
					if (exname.indexOf(name1.value) != -1)
						str += '<option value="o' + i + '">' + exname + '<'+
'/option>\n';
				}
			// Copy the str to list1.innerHTML:
			list1.innerHTML = str;
		}

		function choose(){
			// The chosen name (in the name1 variable)
			var name1 = list1.options[list1.selectedIndex].text;
			// Find the name in the array
			for (i = 0; i < examinees.length; i++)
				if (examinees[i][0] == name1)
					break;
			// The data must be filled in, based on the data of the array
			var idate = document.getElementById('idate');
			idate.innerHTML = examinees[i][1] + " " + examinees[i][2];
			var ivenue = document.getElementById('ivenue');
			ivenue.innerHTML = examinees[i][3];
			// Display the data: makes the DIV block visible
			var writtenexam = document.getElementById('writtenexam');
			writtenexam.style.display = 'block';
		}
	</script>
</head>
```

```
<body>
	<p>Enter a few letters of the name (anywhere within the name), <br />then select one from the list</p>
	Name:<br /><input type="text" id="name1" onKeyUp="typing();"></input>
	<br />
	<select size="10" id="list1" onChange="choose();">
	</select>
	<div id="writtenexam" style="display: none">
		<h2>Written exam</h2>
		<p>
			Date: <span id="idate"></span><br>
			Venue: <span id="ivenue"></span>
		</p>
	</div>
</body>
</html>
```

Add new elements to the DOM, createElement, appendChild

Az oldal kezdetben tartalmazzon egy bekezdést. Az oldal betöltődése után adjunk hozzá még egy bekezdést és egy listát 2 elemmel.

JavaScript nélkül:
Bekezdés1

JavaScript-el:

Bekezdés1

Bekezdés2

- elem1
- elem2

exercise-12.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>exercise</title>
  <script type="text/javascript">
    function add() {
       this.paragraph2 = document.createElement("p");
       this.paragraph2.innerHTML="Paragraph2";
       document.body.appendChild(this.paragraph2);
       this.list1=document.createElement("ul");
       document.body.appendChild(this.list1);
       this.elem=document.createElement("li");
       this.elem.innerHTML="elem1";
       this.list1.appendChild(this.elem);
       this.elem=document.createElement("li");
       this.elem.innerHTML="elem2";
       this.list1.appendChild(this.elem);
    }
```

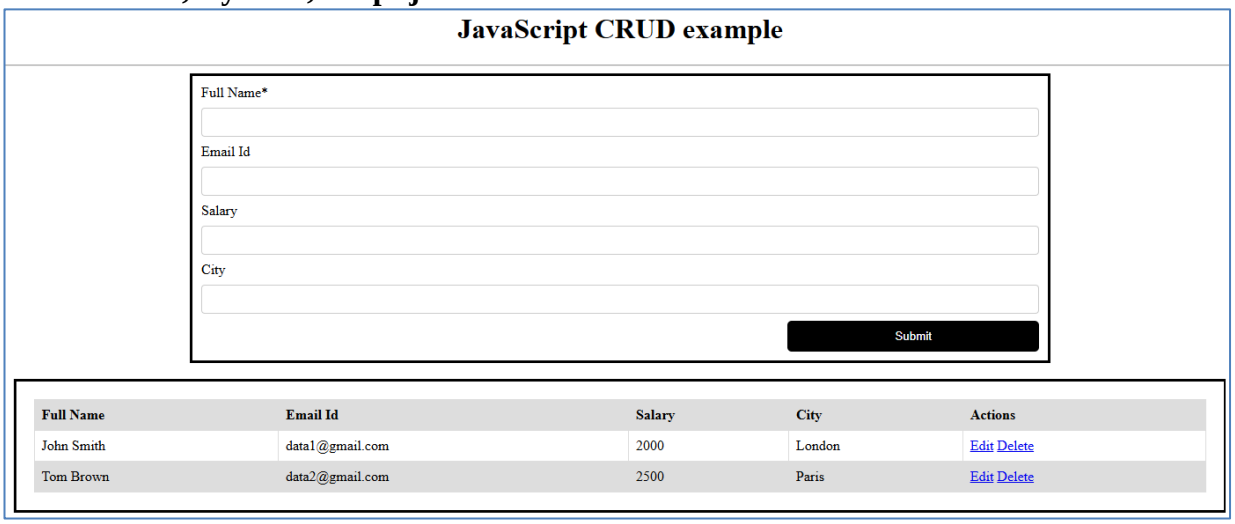
```
// onload event: is activated when the page is fully loaded and the DOM has been built:
    window.onload = function() {
        add();
    };
  </script>
</head>
<body>
  <p>
    Paragraph1
  </p>
</body>
</html>
```

CRUD application – with an Array - The application restarts when the page is refreshed => AJAX-Fetch API will solve this

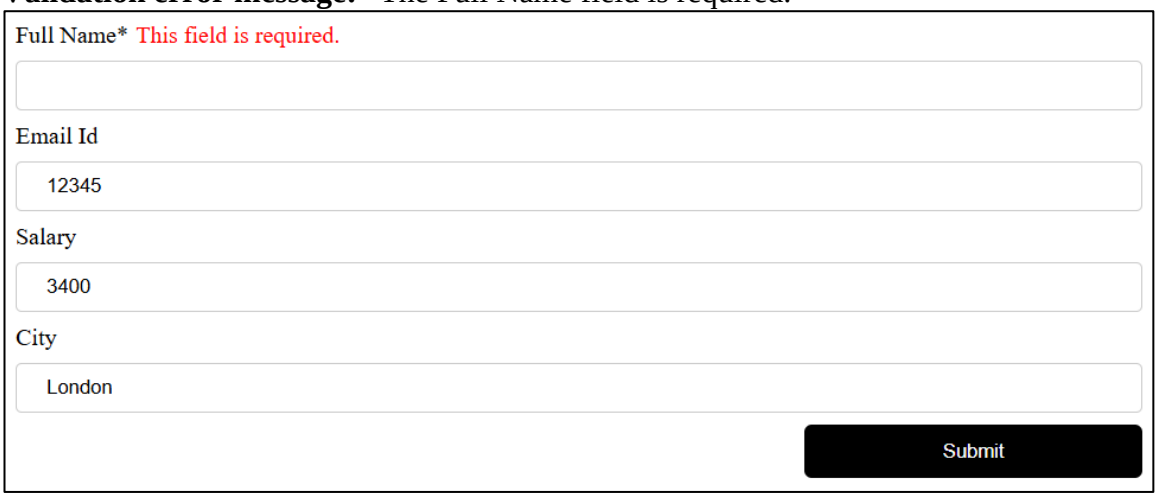
https://www.freecodecamp.org/news/learn-crud-operations-in-javascript-by-building-todo-app/
https://www.javaguides.net/2020/11/javascript-crud-example-tutorial.html

Create the following CRUD application. **CRUD**: Create, Read, Update, Delete

exercise.html, style.css, script.js



Validation error message: The Full Name field is required.



We store the data in an array.
There may be identical records in the array. If we want to avoid this, we need to add a unique ID to each element.

exercise.html

```
<!DOCTYPE html>
<html>
<head>
   <title>JavaScript CRUD example</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
   <h1><center>JavaScript CRUD example</center></h1>
   <hr>
   <div class="employee-form">
// preventDefault(): disables the default event handler assigned to the button click on the form.
//         prevents page reloading
// When the button is clicked, it runs the JS onFormSubmit() function:
      <form onsubmit="event.preventDefault();onFormSubmit();">
         <div>
// Displays the error message This field is required in case of validation error
//         By default the following is hidden: <label class="validation-error hide"…
//         check: script.js: validate() method
            <label>Full Name*</label><label class="validation-error hide"
id="fullNameValidationError">This field is required.</label>
            <input type="text" name="fullName" id="fullName">
         </div>
         <div>
            <label>Email Id</label>
            <input type="text" name="email" id="email">
         </div>
         <div>
            <label>Salary</label>
            <input type="text" name="salary" id="salary">
         </div>
         <div>
            <label>City</label>
            <input type="text" name="city" id="city">
         </div>
         <div  class="form-action-buttons">
            <input type="submit" value="Submit">
         </div>
      </form>
   </div>
   <br/>
// Creates the table structure:
   <div class = "employees-table">
         <table class="list" id="employeeList">
            <thead>
               <tr>
                  <th>Full Name</th>
                  <th>Email Id</th>
                  <th>Salary</th>
```

```
                <th>City</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>

        </tbody>
    </table>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
var selectedIndex = null;
var array1 = new Array();      // We store the data in this array
array1.push({"fullName":"John Smith","email":"data1@gmail.com","salary":"2000","city":"London"});
//OR:
//array1[0]= {"fullName":"John Smith","email":"data1@gmail.com","salary":"2000","city":"London"};
array1.push({"fullName":"Tom Brown","email":"data2@gmail.com","salary":"2500","city":"Paris"});
printArray();

// Print the array into the table below:
function printArray(){
  var table = document.getElementById("employeeList").getElementsByTagName('tbody')[0];
// Empty the existing table:
  table.innerHTML="";
  var newRow;
  for (i = 0; i < array1.length; i++) {
// Inserts a new row into the table:
    newRow = table.insertRow(table.length);
// Inserts an empty cell into the new row:
    cell1 = newRow.insertCell(0);
// Inserts the fullName data from the array into the cell:
    cell1.innerHTML = array1[i].fullName;
    cell2 = newRow.insertCell(1);
    cell2.innerHTML = array1[i].email;
    cell3 = newRow.insertCell(2);
    cell3.innerHTML = array1[i].salary;
    cell4 = newRow.insertCell(3);
    cell4.innerHTML = array1[i].city;
    cell4 = newRow.insertCell(4);
// adds two links to the end of the row: Edit, Delete
//      Pass the index of the selected row as a parameter to the onEdit and onDelete functions:
    cell4.innerHTML = '<a onClick="onEdit('+i+')">Edit</a>' + '<a
onClick="onDelete('+i+')">Delete</a>';
  }
}

// If the user fills out the form and clicks the Submit button:
function onFormSubmit() {
// validate(): checks that the form has been filled out correctly
```

```
    if (validate()) {
// Reads the data from the input fields and returns it in the formData list.
        var formData = readFormData();
// If selectedIndex == null: insert the new record after clicking the button
//         otherwise modify the selected record.
//         selectedIndex is null by default, will get a value on Update
        if (selectedIndex==null)
            insertNewRecord(formData);
        else
            updateRecord(formData);
        resetForm();
    }
}

// Reads the data from the input fields and returns it in a list
function readFormData() {
    var formData = {};
    formData["fullName"] = document.getElementById("fullName").value;
    formData["email"] = document.getElementById("email").value;
    formData["salary"] = document.getElementById("salary").value;
    formData["city"] = document.getElementById("city").value;
    return formData;
}

// Inserts the new row at the end of the array and puts two links at the end of the row: Edit, Delete
//         data: form data in a list
function insertNewRecord(data) {
// Put the new record at the end of the array:
    array1.push({"fullName":data.fullName,"email":data.email,"salary":data.salary,"city":data.city});
// OR: array1[array1.length]=
// {"fullName":data.fullName,"email":data.email,"salary":data.salary,"city":data.city};
    printArray();
}

// Clears input fields. e.g. call it after submitting data.
function resetForm() {
    document.getElementById("fullName").value = "";
    document.getElementById("email").value = "";
    document.getElementById("salary").value = "";
    document.getElementById("city").value = "";
    selectedIndex=null;
}

// The selected record is loaded into the top form for editing:
function onEdit(index) {
    document.getElementById("fullName").value = array1[index].fullName;
    document.getElementById("email").value = array1[index].email;
    document.getElementById("salary").value = array1[index].salary;
    document.getElementById("city").value = array1[index].city;
    selectedIndex=index;
}

// Modifies the selected record with the retrieved form data:
```

```
//        Called on Update
function updateRecord(formData) {
    array1[selectedIndex].fullName=formData.fullName;
    array1[selectedIndex].email=formData.email;
    array1[selectedIndex].salary=formData.salary;
    array1[selectedIndex].city=formData.city;
    printArray();
}

function onDelete(index) {
    if (confirm('Are you sure to delete this record ?')) {
        array.splice(index, 1); // Deleting the entry with the specified index
        resetForm();
        printArray();
    }
}

// validation: The Full Name field is required.
function validate() {
    isValid = true;
// If the Full Name field is empty (validation error):
    if (document.getElementById("fullName").value == "") {
        isValid = false;
// Displays the hidden error message:
        document.getElementById("fullNameValidationError").classList.remove("hide");
    } else {
// if there is no validation error (Full Name field is filled in)
        isValid = true;
// If the validation error message is not hidden, hide it:
        if (!document.getElementById("fullNameValidationError").classList.contains("hide"))
            document.getElementById("fullNameValidationError").classList.add("hide");
    }
    return isValid;
}
```

We dont't deal with CSS here.

style.css

```
.employee-form {
    border-style: solid;
    /* margin-bottom: 10px; */
    /* margin-left: 10px; */
    padding: 10px;
    /* width: 50%; */
    margin: auto;
    width: 50%;
    /* border: 3px solid green; */
    /* padding: 10px; */
}
.employees-table {
    border-style: solid;
    /* margin-bottom: 10px; */
    /* margin-left: 10px; */
```

```
    padding: 20px;
    /* width: 50%; */
    margin: auto;
    width: 70%;
    /* border: 3px solid green; */
    /* padding: 10px; */
}
body > table{
    width: 80%;
}
table{
    border-collapse: collapse;
}
table.list{
    width:100%;
}
td, th {
    border: 1px solid #dddddd;
    text-align: left;
    padding: 8px;
}
tr:nth-child(even),table.list thead>tr {
    background-color: #dddddd;
}
input[type=text], input[type=number] {
    width: 100%;
    padding: 8px 20px;
    margin: 8px 0;
    display: inline-block;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}
input[type=submit] {
    width: 30%;
    background-color: black;
    color: white;
    padding: 10px 18px;
    /* margin: 0px 0; */
    border: none;
    border-radius: 5px;
    cursor: pointer;
}
form div.form-action-buttons{
    text-align: right;
}
a{
    cursor: pointer;
    text-decoration: underline;
    color: #0000ee;
    margin-right: 4px;
}
label.validation-error{
```

```
    color:   red;
    margin-left: 5px;
}
.hide{
    display:none;
}
```

Asynchronous JavaScript

https://www.jsclub.hu/szinkron-es-aszinkron-javascript/

Synchronous operation

exercise-14a.html

```
    <script type="text/javascript">
        window.onload = function() {
            document.writeln("Hello1<br>");
            document.writeln("Hello2<br>");
            document.writeln("Hello3<br>");
        }
    </script>
```

It prints in this order:
Hello1
Hello2
Hello3

The operation is synchronous even if the operation takes a longer time:

exercise-14b.html

```
    <script type="text/javascript">
        window.onload = function() {
            document.writeln("Hello1<br>");
            document.writeln(TimeConsumingOperation(1000000000)+"<br>");
            document.writeln("Hello2<br>");
        }
        function TimeConsumingOperation(a){
            let b=0;
            for (let i = 1; i <= a; i++)
                b+=i;
            return b;
        }
    </script>
```

It prints in this order:
Hello1
500000000067109000
Hello2

Asynchronous operation

exercise-14c.html

```
<script type="text/javascript">
    window.onload = function() {
        document.writeln("Hello1<br>");
        setTimeout(function(){
            document.writeln("Hello2<br>");
        }, 0);
        document.writeln("Hello3<br>");
    }
</script>
```

It prints in this order:
Hello1
Hello3
Hello2

The setTimeout time interval doesn't matter. Same with a 1 second wait:

```
        setTimeout(function(){
            document.writeln("Hello2<br>");
        }, 1000);
```

Techniques for ensuring asynchronous operation - setTimeout, Promise, async/await, fetch, event handler, Web Worker

It will be asynchronous if it contains any of the following (no callbacks among them!):

- setTimeout
- Promise
- async/await
- fetch
- event handler e.g. button click event
- Web Worker

We will study all of them.

A callback is not asynchronous in itself => look at synchronous callback

callback, async, await – they play an important role in Node.js

https://medium.com/womenintechnology/callbacks-vs-promises-vs-async-await-detailed-comparison-d1f6ae7c778a
Callbacks, promises and async/await, these are the methods to handle asynchronous behaviour in javascript. We need asynchronous programming for fetching data from the server, uploading files and handling user interactions. Initially, we used callbacks but it had consequences like deep nesting and code complexity. To tackle this scenario, concepts like promises and async/await were introduced. These concepts helped in providing readable and clean code.

callback - The traditional approach using functions passed as arguments.

synchronous callback - A callback in itself is not asynchronous!!!

https://www.w3schools.com/JS//js_callback.asp
A callback function is a function passed as an argument into another function.

A callback function is intended to be executed later.
Later is typically when a specific event occurs or an asynchronous operation completes.

Using a callback, you could call the calculator function (**myCalculator**) with a callback (**myCallback**), and let the calculator function run the callback after the calculation is finished:

exercise-15a.html

```
<script type="text/javascript">
   function myDisplayer(some) {
      document.write(some);
   }
   function sum(num1, num2, myCallback) {
      myCallback(num1 + num2);
   }
   window.onload = function() {
      document.writeln("Hello1<br>");
      sum(5, 5, myDisplayer);
      document.writeln("<br>Hello2<br>");
   }
</script>
```

It prints (synchronous!)
Hello1
10
Hello2

asynchronous callback – with setTimeout

exercise-15b.html

```
<script type="text/javascript">
   function myDisplayer(some) {
      document.write(some);
   }
   function sum(num1, num2, myCallback) {
      setTimeout(() => {
         myCallback(num1 + num2);
      }, 0);
   }
   window.onload = function() {
      document.writeln("Hello1<br>");
      sum(5, 5, myDisplayer);
      document.writeln("Hello2<br>");
   }
</script>
```

It prints (asynchronous!)
Hello1
Hello2
10

Problem with Callback
https://www.geeksforgeeks.org/javascript/what-is-callback-hell-in-javascript/

Callback Hell happens when multiple dependent asynchronous callbacks are nested, leading to complex and hard-to-manage code that reduces readability and maintainability.

https://www.freecodecamp.org/news/javascript-promise-object-explained/

exercise-15c.html

```
<script type="text/javascript">
   function stepOne(value, callback) {
      setTimeout(() => {
         document.write(value);
         callback();
      }, 1000);
   }
   function stepTwo(value, callback) {
      setTimeout(() => {
         document.write(value);
         callback();
      }, 1000);
   }
   function stepThree(value, callback) {
      setTimeout(() => {
         document.write(value);
         callback();
      }, 1000);
   }
   // Run the functions sequentially with callbacks
   stepOne(1, () => {
      stepTwo(2, () => {
         stepThree(3, () => {
            setTimeout(() => {
               document.write("All steps completed.");
            }, 1000);
         });
      });
   });
</script>
```

=> 123All steps completed.

This code pattern is hard to read and it's messy.

Promises: A better alternative that improves readability and avoids callback nesting.

https://medium.com/womenintechnology/callbacks-vs-promises-vs-async-await-detailed-comparison-d1f6ae7c778a
https://www.w3schools.com/js/js_promise.asp
https://www.geeksforgeeks.org/javascript/callbacks-vs-promises-vs-async-await/
Promises offer a more structured approach to handle asynchronous operations, addressing the callback hell problem. They represent the eventual completion (or failure) of an asynchronous task.

Promise States

A promise can be in one of three exclusive states:

- Pending: The initial state; the operation has started but is neither fulfilled nor rejected.

- Fulfilled: The operation completed successfully, and a value is available.
- Rejected: The operation failed, and a reason (error) is available.

exercise-16a.html

```
<script type="text/javascript">
   // Function to fetch user data with a Promise
   function fetchData() {
      return new Promise((resolve, reject) => {
         if (true)
            resolve("Task completed successfully!");
         else
            reject("Task failed");
       });
   }
   window.onload = () => {
      document.write("Hello1<br>");
      fetchData()
         .then((a) => a+" Hello world!")
         .then((b) => document.write("Success: ", b))
         .catch((error) => document.write("Error:", error));
      document.write("Hello2<br>");
   }
</script>
```

It prints:
Hello1
Hello2
Success: Task completed successfully! Hello world!

In the above code, we have a fetchData() function which returns a Promise. If the promise is resolved, the result will be displayed and if it is rejected then the catch block will be executed which will display the error.

Promise chain

Solve our previous **Callback Hell** exercise with **Promise chain**.

https://www.w3schools.com/js/js_async.asp

exercise-16b.html

```
<script>
   function step1() {
      document.write("X<br>");
      return Promise.resolve("A");
   }
   function step2(value) {
      document.write("Y<br>");
      return Promise.resolve(value + "B");
   }
   function step3(value) {
      document.write("Z<br>");
      return Promise.resolve(value + "C");
   }
```

```
    window.onload = () => {
        document.write("Hello1<br>");
        step1()
        .then((value) => {
            return step2(value);
        })
        .then((value) => {
            return step3(value);
        })
        .then((value) => {
            myDisplayer(value);
        });
        document.write("Hello2<br>");
    }
    function myDisplayer(some) {
        document.write(some+"<br>");
    }
</script>
```

The chain runs step by step as each promise finishes.

It prints:
Hello1
X
Hello2
Y
Z
ABC

If you want it to print Hello2 at the end, you need to put that in the then section as well:

```
    window.onload = () => {
        document.write("Hello1<br>");
        step1()
        .then((value) => {
            return step2(value);
        })
        .then((value) => {
            return step3(value);
        })
        .then((value) => {
            myDisplayer(value);
        })
        .then(() => {
            document.write("Hello2<br>");
        });
    }
```

It prints:
Hello1
X
Y
Z

ABC
Hello2

Here, you can see that each function returns a promise. The function calls using then() methods maintain a clear order of steps.
In a real-world project where you have many lines of code inside the callback functions, using Promises provides a massive gain in your ability to read, extend, and maintain the code.

https://www.freecodecamp.org/news/asynchronous-javascript-explained/
Promises are a neat way to fix problems brought about by callback hell, in a method known as promise chaining. You can use this method to sequentially get data from multiple endpoints, but with less code and easier methods.

But there is an even better way! You might be familiar with the following method, as it's a preferred way of handling data and API calls in JavaScript:

Async/Await: A modern and cleaner syntax that makes asynchronous code look synchronous.

https://www.geeksforgeeks.org/javascript/callbacks-vs-promises-vs-async-await/
Provides a more synchronous-like flow, enhancing readability.
Enables writing asynchronous code in a synchronous manner, simplifying control flow.
Async/Await is built on top of Promises and allows asynchronous code to be written in a synchronous style, making it easier to read and understand.

https://www.w3schools.com/js/js_async.asp
async and await make promises easier
You still use promises, but you write the code like normal step by step code.
async makes a function return a Promise
await makes a function wait for a Promise

The async Keyword

The async keyword before a function makes the function return a promise.

```
async function myFunction() {
  return "Hello";
}
```

Is the same as:

```
function myFunction() {
  return Promise.resolve("Hello");
}
```

The result is handled with **then()** because it is a promise:

exercise-17a.html

```
<script>
   async function myFunction() {   // Create a promise
      return "Hello";
   }
   // Handle promise result:
   myFunction().then((value) => {myDisplayer(value);});
   function myDisplayer(some) {
```

```
    document.write(some+"<br>");
  }
</script>
```

It prints:
Hello

The await Keyword

The await keyword makes a function pause the execution and wait for a resolved promise before it continues:

exercise-17b.html

```
<script>
  async function myFunction() {   // Create a promise
    return "Hello";
  }
  async function run() {
    let value = await myFunction();
    myDisplayer(value);
  }
  run();
  function myDisplayer(some) {
    document.write(some+"<br>");
  }
</script>
```

Solve the previously studied **Promise chain** task with **Async/Await** elements

exercise-17c.html

```
<script>
  function step1() {
    document.write("X<br>");
    return Promise.resolve("A");
  }
  function step2(value) {
    document.write("Y<br>");
    return Promise.resolve(value + "B");
  }
  function step3(value) {
    document.write("Z<br>");
    return Promise.resolve(value + "C");
  }
  window.onload = () => {
    document.write("Hello1<br>");
    async function run() {
      let v1 = await step1();
      let v2 = await step2(v1);
      let v3 = await step3(v2);
      myDisplayer(v3);
    }
    run();
    document.write("Hello2<br>");
  }
```

```
    function myDisplayer(some) {
        document.write(some+"<br>");
    }
</script>
```

It prints:
Hello1
X
Hello2
Y
Z
ABC

3-4-React basics - JavaScript library-framework

Some call React a library, some call it a framework.

Introduction – Only informative text

Some React descriptions:
https://reactjs.org/tutorial/tutorial.html
https://www.w3schools.com/REACT/default.asp
https://react-tutorial.app/
https://www.javatpoint.com/reactjs-tutorial
https://www.tutorialspoint.com/reactjs/index.htm
https://www.newline.co/fullstack-react/
https://ibaslogic.com/react-tutorial-for-beginners/
https://www.freecodecamp.org/news/react-tutorial-build-a-project/

- React is a JavaScript library for building user interfaces.
- React is used to build single-page applications.
- React allows us to create reusable UI components.

React vs JavaScript – Advantages of React – Logic of React (components, we don't have to deal with the DOM,...) - React is better for larger applications

Where is JavaScript better?
- simple website
- few interactions
- small script (e.g. button press)

Where is React better?
- Complex user interfaces
- many state changes
- large application

React is better for large applications and complex user interfaces
- If your page contains many interactive elements whose state changes frequently (such as a Facebook news feed), React's declarative approach makes the code more transparent.
- We break the interface into components - It is easier to reuse elements
  If you need to use many similar elements (buttons, cards, forms) repeatedly throughout your project, React's component-based architecture speeds up development and makes maintenance easier.
  In JavaScript, this can quickly lead to repetitive and messy code.
- Declarative Programming
  - JS (Imperative): You have to write down the exact steps: "Find the button, add the event handler, change the color, rewrite the text."
  - **React (Declarative):** You just tell it how the interface should look in a given state: "If the button is pressed, it should be red." **And React does the background work.**
- **We don't have to deal with the DOM, React uses different logic**
  We don't use in components: id, getElementById, getElementsByClassName

- **No need to manually modify the DOM: React solves the modification at the background**

React uses Virtual DOM technology, which only updates the parts of the browser that have actually changed. This can be a significant performance advantage for complex, data-driven interfaces.
Modifying the browser's DOM directly is a slow process.
React uses a "Virtual DOM". It first computes changes in memory and only sends the bare minimum to the browser, which significantly improves performance for large applications.

- User interface automatically updates on state changes

In JavaScript, when a piece of data changes, you have to manually find all the affected HTML elements and update them.
Managing state in React is much cleaner than manually manipulating the DOM in JavaScript. When the "state" changes, React automatically redraws only the part of the interface that has changed. You don't have to hunt for DOM elements for us.

- JavaScript code quickly becomes difficult to understand in large projects.

https://www.geeksforgeeks.org/reactjs/difference-between-javascript-and-reactjs/
https://www.pulsion.co.uk/blog/react-vs-javascript/
https://www.simplilearn.com/react-vs-javascript-article
https://eluminoustechnologies.com/blog/react-vs-javascript/
https://www.dhiwise.com/post/vanilla-javascript-vs-react-whats-right-for-developers

- React.js is a **popular JavaScript library** for building user interfaces, **developed by Facebook**.
- It emphasizes **component-based architecture** and enables developers to **efficiently create large-scale applications with reusable UI components**.
- It allows developers to **create reusable UI components** that can **efficiently update and render when data changes.**
- React uses a **virtual DOM for better performance**.
- **Component-Based Architecture**: **UIs are composed of reusable components**, each managing its own state. **Component Reusability: Encourages reusable UI components**, **saving development time.**
- **Virtual DOM**: **Enhances performance by updating only the necessary DOM elements.**
- **JSX**: **Allows writing HTML-like syntax within JavaScript, improving code readability.**

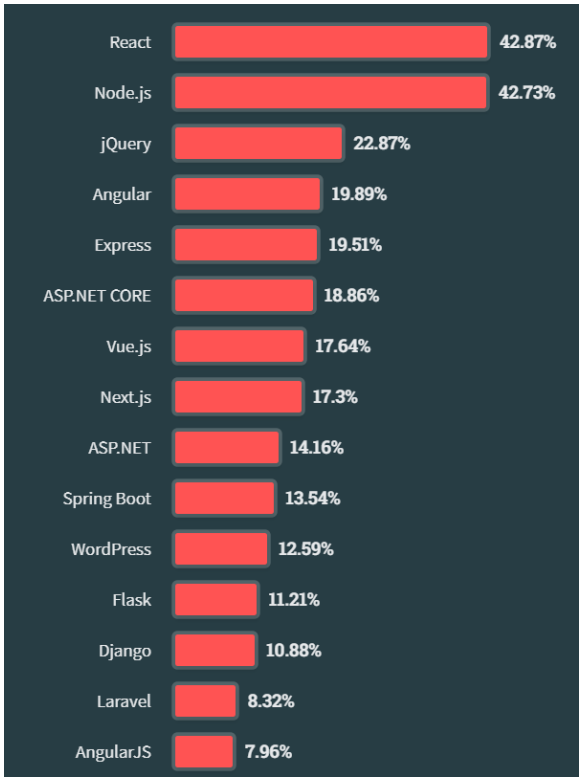
When should you not use React?
If you only need a simple, few-line script (e.g. for a simple form), React would be unnecessarily large and complicated - in that case, plain JavaScript is the better choice.

The browser does not understand React, so a compiler is needed to convert React code into JavaScript.

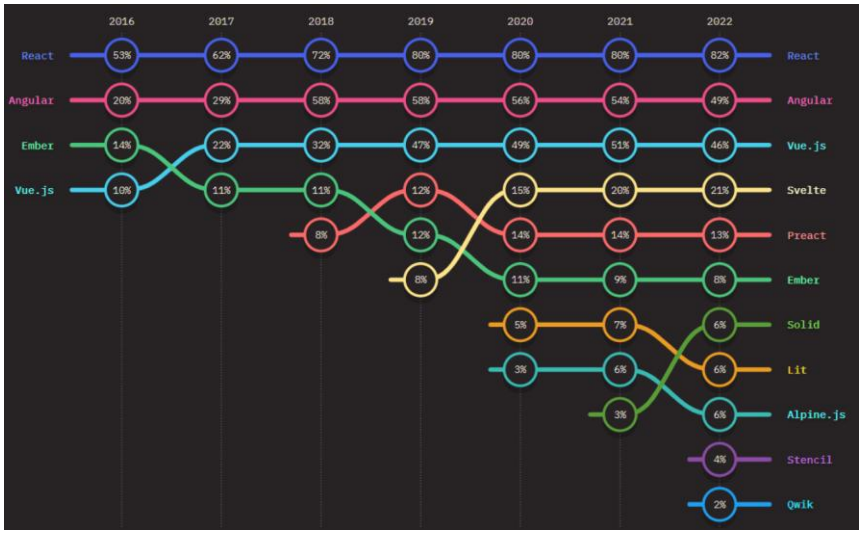
Modern web browsers (like Chrome, Firefox, or Safari) can only directly interpret standard JavaScript (ECMAScript), HTML, and CSS.

Why React? - React vs Angular vs Vue

https://survey.stackoverflow.co/2023/#most-popular-technologies-webframe-prof
How many of those surveyed used:

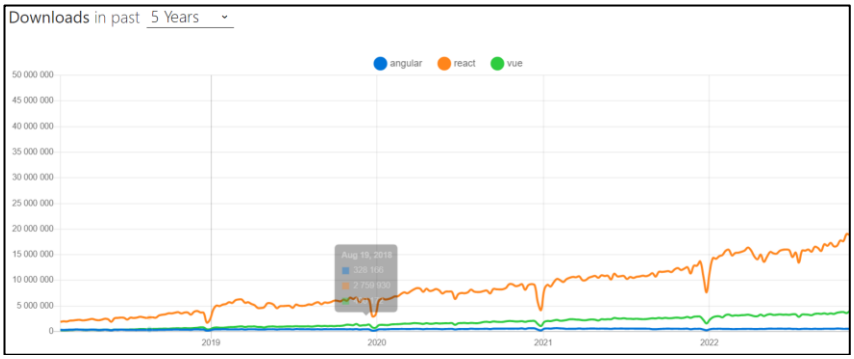


https://www.lambdatest.com/blog/best-javascript-frameworks/

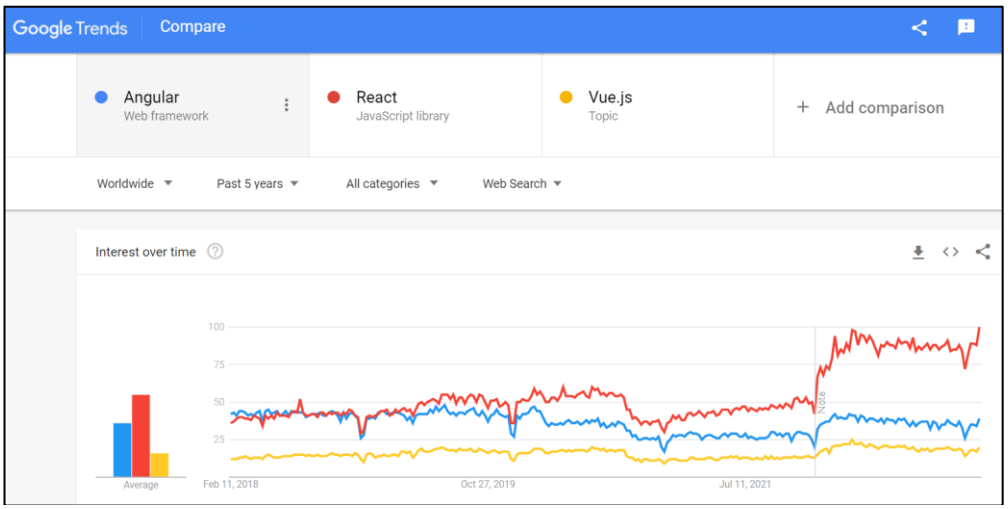


NPM trends

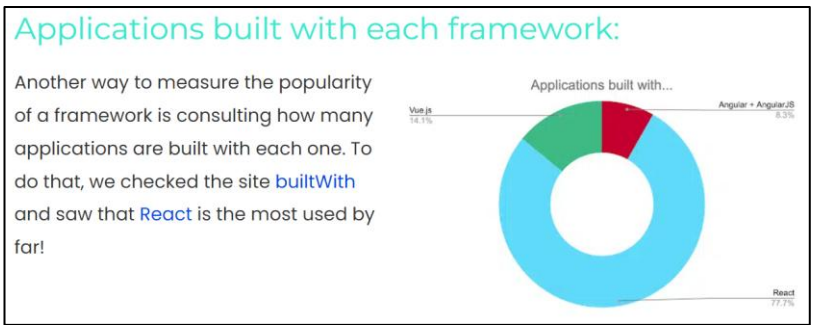
https://npmtrends.com/angular-vs-react-vs-vue

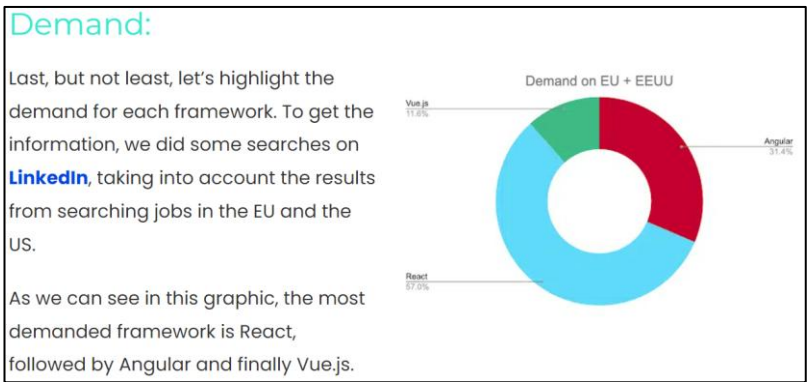


Google trends
https://trends.google.com/trends/explore?q=%2Fg%2F11c6w0ddw9,%2Fm%2F012l1vxv,%2Fg%2F11c0vmgx5d



https://www.zartis.com/angular-vs-react-vs-vuejs/





React versions

https://react.dev/versions

19.2.0 (October, 2025)
19.1.0 (March, 2025)
19.0.0 (December, 2024)
18.0.0 (March 2022)
17.0.0 (October 2020)
16.0 (September 2017)
15.0.0 (April 2016)
14.0 (October 2015)
13.0 (March 2015)
12.0 (October 2014)

https://www.lambdatest.com/blog/best-javascript-frameworks/

Prominent Websites Built with React:
Airbnb, Asana, BBC, Cloudflare, Codecademy, Dropbox, Facebook, GitHub, Imgur, Instagram, Medium, Netflix, OkCupid, Paypal, Periscope, Pinterest, Product Hunt, Reddit, Salesforce, Scribd, Shopify, Slack, Snapchat, Squarespace, Tesla, The New York Times, Typeform, Twitter, Uber, Udemy, WhatsApp, Zendesk.

Prominent Websites Built with Angular:
Google, Allegro, Blender, Clickup, Clockify, Delta, Deutsche Bank, DoubleClick, Freelancer, Forbes, Guardian, IBM, Instapage, iStock, JetBlue, Lego, Mailerlite, Microsoft Office, Mixer, Udacity, Upwork, Vevo, Walmart, Weather, WikiWand, Xbox, YouTube.

Prominent Websites Built with Vue.js:
9gag, Adobe, Apple Swift UI, Behance, Bilibili, BMW, Chess, Font Awesome, GitLab, Hack The Box, Laravel, Laracasts, Louis Vuitton, Namecheap, Netlify, Netguru, Nintendo, Pluralsight, Shien, Stack Overflow, Trivago, Trustpilot, Upwork, Wizzair, Zoom.

Basic exercises – with Babel/standalone (React compiler) - compiles at runtime, therefore slower

The browser understands JavaScript, but does not understand React => We need a compiler:
Babel/standalone

https://babeljs.io/

Babel is a toolchain that is mainly used to convert ECMAScript 2015+ code into a backwards compatible version of JavaScript in current and older browsers or environments.

https://babeljs.io/docs/en/babel-standalone

@babel/standalone provides a standalone build of Babel for use in browsers and other non-Node.js environments.

Babel, JSX and React

Babel can convert JSX syntax!

JSX: JSX stands for JavaScript XML. JSX allows us to write HTML in React.

JSX converts HTML tags into react elements.

Here we are still using files with HTML extension, in the next chapter we will use files with **JSX** extension.

1-Rendering HTML code into a DIV with CSS.

Name
John
Elsa

exercise-01.html

```
<!DOCTYPE html>
<html>
 <head>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>     <style>
                table, th, td {
                border: 3px solid blue;
                border-collapse: collapse;
                }
        </style>
 </head>
 <body>
        <div id="root"></div>
        <script type="text/babel">
                const myelement = (
                  <table>
                        <tr>
                          <th>Name</th>
                        </tr>
                        <tr>
                          <td>John</td>
                        </tr>
                        <tr>
                          <td>Elsa</td>
                        </tr>
                  </table>
                );
                ReactDOM.render(myelement, document.getElementById('root'));
        </script>
 </body>
</html>
```

ReactDOM: ReactDom is a core react package that provides methods to interact with the Document Object Model or DOM.
render: renders a piece of JSX (“React node”) into a browser DOM node.
render: ad, nyújt

Right click on page
- Page source
- Inspect

2- constants, variables, fragment, className, CSS

exercise-02.html

In the previous example, the 3 lines in the <head> section will be the same in each example, so I've labeled them ……. from now on:

```
<!DOCTYPE html>
<html>
  <head>
……..
    <style>
       .myclass {color: blue;}
    </style>
  </head>
  <body>
          <div id="root1"></div>
          <div id="root3"></div>
          <div id="root4"></div>
          <div id="root5"></div>
          <div id="root6"></div>
          <script type="text/babel">
                    const myElement1 = <h1>React is {5 + 5} times better with JSX</h1>;
                    const root1 = ReactDOM.createRoot(document.getElementById('root1'));
                    root1.render(myElement1);

                    const myElement3 = (
// Multiple elements must be placed in ONE container element, e.g. a DIV,
//         which we use to connect them to the DOM.
                      <div>
                              <p>I am a paragraph.</p>
                              <p>I am a paragraph too.</p>
                      </div>
                    );
                    const root3 = ReactDOM.createRoot(document.getElementById('root3'));
                    root3.render(myElement3);

                    const myElement4 = (
// Alternatively, you can use a fragment to organize multiple elements into a unit.
                      <>
                              <p>I am a paragraph.</p>
                              <p>I am a paragraph too.</p>
                      </>
                    );
                    const root4 = ReactDOM.createRoot(document.getElementById('root4'));
                    root4.render(myElement4);

                    // All items must be closed:
                    const myElement5 = <input type="text" />;            ehelyett: <input type="text">
                    const root5 = ReactDOM.createRoot(document.getElementById('root5'));
                    root5.render(myElement5);

// In HTML we write it like this: <h1 class="myclass">, but since class is a reserved word in JavaScript,
```

```
//        we write it like this:   <h1 className="myclass">
                const myElement6 = <h1 className="myclass">Hello World</h1>;
                const root6 = ReactDOM.createRoot(document.getElementById('root6'));
                root6.render(myElement6);
        </script>
  </body>
</html>
```

createRoot: lets you create a root to display React components inside a browser DOM node.

3-Components, Props

https://www.w3schools.com/REACT/react_components.asp
Components are like **JavaScript functions** that **return HTML elements**. Components are **independent and reusable bits of code**.

Two types of Components

- Function component
- Class component

https://www.w3schools.com/REACT/react_props.asp
Props: function's arguments: They contain key-value pairs e.g. color="red"

Function component – Function vs. Class components - It was introduced later than the class component, but is now more widespread.

Functional components are recommended for most new development due to their simplicity, performance, and flexibility.
Looking forward, function components with Hooks are likely to become the standard way of building React applications. They're easier to use and offer better performance, making them a favorite among developers.
While class components still have their place, especially in older projects, the trend is moving towards function components with Hooks.
Use functional components when you build new components. Refactor the class components to function components when you have time or you're fixing bugs in them.
It is now suggested to use Function components along with Hooks, instead of Class components.

https://www.freecodecamp.org/news/function-component-vs-class-component-in-react/
https://www.daggala.com/function-components-vs-class-components-in-react/

https://www.w3schools.com/react/react_components.asp
https://www.twilio.com/en-us/blog/react-choose-functional-components
https://www.geeksforgeeks.org/blogs/differences-between-functional-components-and-class-components/

exercise-03.html

```
<!DOCTYPE html>
<html>
  <head>
……..
  </head>
  <body>
        <div id="root1"></div>
        <div id="root2"></div>
        <div id="root3"></div>
        <div id="root4"></div>
        <script type="text/babel">
// Function Component, returns HTML:
                function Car() {
                  return <h2>Hi, I am a Car!</h2>;
                }
                const root1 = ReactDOM.createRoot(document.getElementById('root1'));
                root1.render(<Car />);

                function Car2(props) {
                  return <h2>I am a {props.color} Car2!</h2>;
                }
                const root2 = ReactDOM.createRoot(document.getElementById('root2'));
                root2.render(<Car2 color="red"/>);

                function Car3() {
                  return <h2>I am a Car3!</h2>;
                }
                function Garage() {
                  return (
                        <>
                          <h1>Who lives in my Garage?</h1>
                          <Car3 />
                        </>
                  );
                }
                const root3 = ReactDOM.createRoot(document.getElementById('root3'));
                root3.render(<Garage />);
```

Class component – we use classes (Object-oriented programming)

```
// Class component
//A class component must include the extends React.Component statement. This statement creates an
// inheritance to React.Component, and gives your component access to React.Component's functions.
                class Car4 extends React.Component {
                  render() {
                        return <h2>Hi, I am a Car4!</h2>;
                  }
                }
```

```
        const root4 = ReactDOM.createRoot(document.getElementById('root4'));
        root4.render(<Car4 />);
    </script>

  </body>
</html>
```

Hi, I am a Car!
I am a red Car2!
Who lives in my Garage?
I am a Car3!
Hi, I am a Car4!

Instead:

```
    const root1 = ReactDOM.createRoot(document.getElementById('root1'));
    root1.render(<Car />);
```

this could also be:

```
    ReactDOM.createRoot(document.getElementById('root1')).render(<Car />);
```

Multi-level Prop – with JSON

feladat-03b.html

```
<!DOCTYPE html>
<html>
  <head>
……..
  </head>
  <body>
    <div id="root3"></div>
    <script type="text/babel">
      function Car3(props) {
        return <h2>I am a { props.brand.model }!</h2>;
      }
      function Garage3() {
        const carInfo = { name: "Ford", model: "Mustang" };          // JSON
        return (
          <>
            <h1>Who lives in my garage?</h1>
            <Car3 brand={ carInfo } />
          </>
        );
      }
      const root3 = ReactDOM.createRoot(document.getElementById('root3'));
      root3.render(<Garage3 />);
    </script>
  </body>
</html>
```

Prints:
Who lives in my garage?
I am a Mustang!

Do not use getElementById and getElementsByClassName selectors inside the component

You can have such an ID or Class outside the component on the webpage.

Don't use IDs inside components

Don't use IDs inside components because they must be unique on the page. React allows it, but we avoid it

4-Events

https://www.w3schools.com/REACT/react_events.asp

Similar to JavaScript events
We have to use camelCase format onClick
Writing React event handlers: onClick={shoot}

Put 2 components on the page. Both components return a button. Clicking the first button will display an alert window: Great Shot! Clicking the second button will also display text in the alert window, but we will provide the text as a parameter.

exercise-04.html

```
<!DOCTYPE html>
<html>
  <head>
……….……
  </head>
  <body>
    <div id="root1"></div>
    <div id="root2"></div>
    <script type="text/babel">
// component:
      function Football1() {
// arrow function:
        const shoot = () => {
          alert("Great Shot!");
        }
        return <button onClick={shoot}>Take the shot!</button>;
      }
      const root1 = ReactDOM.createRoot(document.getElementById('root1'));
      root1.render(<Football1 />);

// Like the previous component, only here the shoot function also has an input parameter:
      function Football2() {
        const shoot = (a) => {
           alert(a);
        }
        return <button onClick={() => shoot("Goal!")}>Take the shot!</button>;
      }
      const root2 = ReactDOM.createRoot(document.getElementById('root2'));
      root2.render(<Football2 />);
    </script>
```

```
  </body>
</html>
```

5-useState Hook – preserving the value and state of variables - Hooks are alternative to classes.

Hooks are an alternative to classes.
They let you use state and other React features without writing a class.
https://react.dev/reference/react/hooks
https://www.w3schools.com/react/react_hooks.asp
https://www.geeksforgeeks.org/reactjs/reactjs-hooks/

We can store and modify the states of variables in components without using a class.
We can set an initial value.
We provide a setter method to set the current value of the variable: count=>setCount
This allows us to preserve the values of variables between function calls. Otherwise, the variables would disappear when the function ends.
e.g. number of clicks: When a button is clicked, the value of the variable is increased by 1.
Hook: variable+initial value+Setter

If the value of the useState variable changes, the component is re-rendered!

Define two components. In the first one, define a useState variable with an initial value of 0. Write the value of the variable on the page. Underneath it, add a button that, when clicked, increases the value of the variable by 1.
In the second component, define a useState variable with an initial value of "red". Underneath it, add a button that, when clicked, changes the value of the variable to "blue".

exercise-5.html

```
<!DOCTYPE html>
<html>
  <head>
……..
  </head>
  <body>
    <div id="root1"></div>
    <div id="root2"></div>
    <script type="text/babel">
      const initialValue = React.useState;
      function Example() {
// defines a new state variable called count. initialValue: 0
        const [count, setCount] = initialValue(0);
        return (
        <div>
          <p>You clicked {count} times</p>
// Increases the count value by 1 when the button is clicked:
          <button onClick={() => setCount(count + 1)}>Click me</button>
```

```
      </div>
      );
    }
    const root1 = ReactDOM.createRoot(document.getElementById('root1'));
    root1.render(<Example />);

    function FavoriteColor() {
      const [color, setColor] = initialValue("red");
      return (
      <>
        <h1>My favorite color is {color}!</h1>
        <button type="button" onClick={() => setColor("blue")}>Blue</button>
      </>
      )
    }
    const root2 = ReactDOM.createRoot(document.getElementById('root2'));
    root2.render(<FavoriteColor />);
  </script>
 </body>
</html>
```

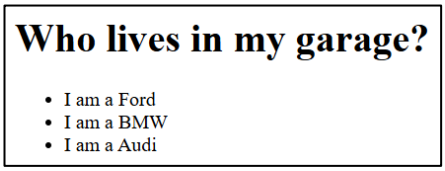
If the component state (useState variable) changes, the component is re-rendered => provides dynamic content in response to user interaction

e.g. here if the count or color states change, then the component is re-rendered, so the new value appears on the page.
By default, when your component's state change, your component will re-render.
https://legacy.reactjs.org/docs/react-component.html

6-Lists, loop, map, Keys

https://www.w3schools.com/REACT/react_lists.asp

We can render lists using loops. To do this, we use the map() Javascript array method.
Each element of the list will be a separate component.

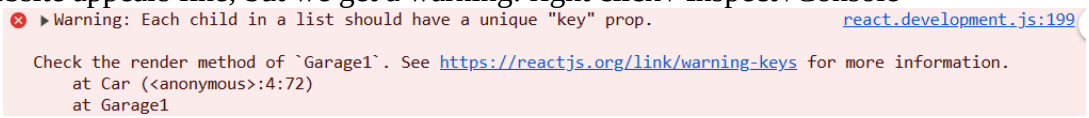


exercise-06a.html

```
<!DOCTYPE html>
<html>
 <head>
……..
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function Car(props) {
    return <li>I am a { props.brand }</li>;
```

```
    }

    function Garage1() {
// cars list
      const cars = ['Ford', 'BMW', 'Audi'];
      return (
      <>
        <h1>Who lives in my garage?</h1>
        <ul>
// For each element (car) in the cars list, call the Car component with the parameter brand={car}
          {cars.map((car) => <Car brand={car} />)}
        </ul>
      </>
      );
    }
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Garage1 />);
  </script>
 </body>
</html>
```

The website appears fine, but we get a warning: right click / Inspect /Console



Warning: Each child in a list should have a unique "key" prop.

Solution => Keys

Keys – list here: a sequence of key-value pairs

Used to identify items in a list.

Helps keep track of items. If an item is updated or deleted, only that item will be re-rendered and not the entire list.

https://www.geeksforgeeks.org/reactjs-keys/

> React JS keys are an important concept for handling dynamic content, especially when rendering lists of elements. Keys help React identify which items have changed, been added, or removed, allowing for efficient updates to the user interface.

We can't access keys within a component (nor do we want to).

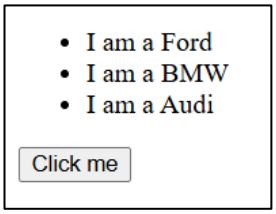
Rewrite the previous example so that we use keys.

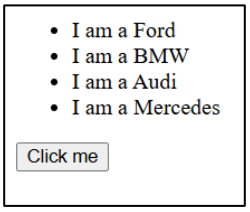
exercise-06b.html

```
<!DOCTYPE html>
<html>
 <head>
……
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function Car(props) {
```

```
    return <li>I am a { props.brand }</li>;
  }
  function Garage() {
    return (
      <>
        <h1>Who lives in my garage?</h1>
        <ul>
          {cars.map((car) => <Car key={car.id} brand={car.brand} />)}
        </ul>
      </>
    );
  }
  const cars = [
    {id: 1, brand: 'Ford'},
    {id: 2, brand: 'BMW'},
    {id: 3, brand: 'Audi'}
  ];
  const root = ReactDOM.createRoot(document.getElementById('root'));
  root.render(<Garage />);
 </script>
</body>
</html>
```

After the list is displayed, modify the list: click a button to add a new item.
To make the component re-render after the list is modified: use a useState state variable that is modified after the list is modified.





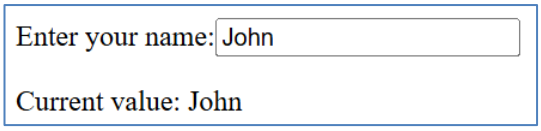
exercise-06c.html

```
<!DOCTYPE html>
<html>
 <head>
……..
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   const initialValue = React.useState;
   function Car(props) {
    return <li>I am a { props.brand }</li>;
   }
   function Garage() {
    const [count, setCount] = initialValue(0);
    return (
      <>
        <h1>Who lives in my garage?</h1>
        <ul>
```

```
          {cars.map((car) => <Car key={car.id} brand={car.brand} />)}
        </ul>
        <button onClick={() =>  {cars.push({id: 5, brand: 'Mercedes'});setCount(count + 1);}}>Click
me</button>
      </>
    );
  }
  const cars = [
    {id: 1, brand: 'Ford'},
    {id: 2, brand: 'BMW'},
    {id: 3, brand: 'Audi'}
  ];
  const root = ReactDOM.createRoot(document.getElementById('root'));
  root.render(<Garage />);
 </script>
</body>
</html>
```

7 – Form

Exercise
Add a text-box to the page. When you type text into the text-box, the text should appear in a paragraph. Change the text displayed for each character typed (onChange event).

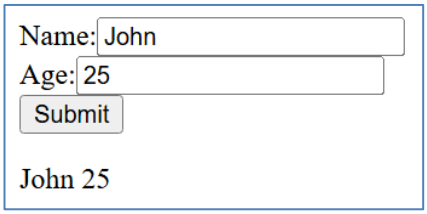


exercise-07a.html

```
<!DOCTYPE html>
<html>
 <head>
…….
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function MyForm() {
    const [name, setName] = React.useState("");
    return (
     <>
      Name:<input type="text" value={name} onChange = {(e) => setName(e.target.value)} />
      <p>Current value: {name}</p>
     </>
    )
   }
   ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
  </script>
 </body>
</html>
```

Exercise

Put 2 input fields and a button on the page. After clicking the button, write the values of the input fields in a paragraph.



We add 1-1 useState variable to store the input fields and the result. We change the value of the useState variable for each character typed in the input field (onChange event). By clicking the button (onClick event), we enter the result into the result useState variable.

exercise-07b.html

```
<!DOCTYPE html>
<html>
  <head>
……….
  </head>
  <body>
    <div id="root"></div>
    <script type="text/babel">
      function MyForm() {
        const [name, setName] = React.useState("");
        const [age, setAge] = React.useState("");
        const [result, setResult] = React.useState("");
        return (
          <>
            Name:<input type="text" onChange = {(e) => setName(e.target.value)} /><br />
            Age:<input type="text" onChange = {(e) => setAge(e.target.value)} /><br />
            <button onClick={(e) => {e.preventDefault(); setResult(name+" "+age);}}>Send</button>
            <p>{result}</p>
          </>
        )
      }
      ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
    </script>
  </body>
</html>
```

e.preventDefault():

e:event,

preventDefault(): Clicking on the button, prevent it from submitting a form and reload the page.

Similar to the previous solution. Here we use a form. By clicking the button, we enter the result useState variable in the form's onSubmit event.

exercise-07c.html

```
<!DOCTYPE html>
<html>
……
  </head>
```

```
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function MyForm() {
    const [name, setName] = React.useState("");
    const [age, setAge] = React.useState("");
    const [result, setResult] = React.useState("");
    return (
     <>
      <form onSubmit={(e) => {e.preventDefault(); setResult(name+" "+age);}}>
       Name:<input type="text" onChange = {(e) => setName(e.target.value)} /><br />
       Age:<input type="text" onChange = {(e) => setAge(e.target.value)} /><br />
       <button>Send</button>
      </form>
      <p>{result}</p>
     </>
    )
   }
   ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
  </script>
 </body>
</html>
```

Instead of:
<button>Send</button>
this is good as well:
<input type="submit" value="Send"/>

Similar to the previous solution. Here, we implement the event handler functions with named functions.

exercise-07d.html

```
<!DOCTYPE html>
<html>
 <head>
……….
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function MyForm() {
    const [name, setName] = React.useState("");
    const [age, setAge] = React.useState("");
    const [result, setResult] = React.useState("");
    function handleChangeName(e) {setName(e.target.value);}
    function handleChangeAge(e) {setAge(e.target.value);}
    function handleSubmit(e) {e.preventDefault(); setResult(name+" "+age);
    }
    return (
     <>
      <form onSubmit={handleSubmit}>
       <label>Name:<input type="text" onChange={handleChangeName} /></label><br />
       <label>Age:<input type="text" onChange={handleChangeAge} /></label><br />
       <button>Send</button>
```

```
        </form>
        <p>{result}</p>
       </>
     )
    }
    ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
  </script>
 </body>
</html>
```

Similar to the previous solution. We read the data of the input fields when the button is clicked, we use **useRef** for this.

useRef:

- When you want a component to “remember” some information, but you don’t want that information to trigger new renders, you can use a ref.
- useRef is a React Hook that lets you reference a value that’s not needed for rendering.

https://react.dev/learn/referencing-values-with-refs
https://react.dev/reference/react/useRef

exercise-07e.html

```
<!DOCTYPE html>
<html>
 <head>
…….
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
   function MyForm() {
    const nameRef = React.useRef();
    const ageRef = React.useRef();
    const [name, setName] = React.useState("");
    const [age, setAge] = React.useState("");
    function handleSubmit(e) {e.preventDefault(); setName(nameRef.current.value);
setAge(ageRef.current.value); }
    return (
     <>
      <form onSubmit={handleSubmit}>
       <label>Name:<input type="text" ref={nameRef} /></label><br />
       <label>Age:<input type="text" ref={ageRef} /></label><br />
       <button>Send</button>
      </form>
      <p>{name} {age}</p>
     </>
   )
  }
  ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
  </script>
 </body>
</html>
```

Similar to the previous solution. Here we read the value of the input field using the **name** attribute, which is common in HTML.

exercise-07f.html

```
<!DOCTYPE html>
<html>
  <head>
……….
  </head>
  <body>
    <div id="root"></div>
    <script type="text/babel">
      function MyForm() {
        const [name, setName] = React.useState("");
        const [age, setAge] = React.useState("");
        function handleSubmit(e) {e.preventDefault(); setName(e.target.nName.value);
setAge(e.target.nAge.value);
        }
        return (
          <>
            <form onSubmit={handleSubmit}>
              <label>Name:<input type="text" name="nName" /></label><br />
              <label>Age:<input type="text" name="nAge" /></label><br />
              <button>Send</button>
            </form>
            <p>{name} {age}</p>
          </>
        )
      }
      ReactDOM.createRoot(document.getElementById('root')).render(<MyForm />);
    </script>
  </body>
</html>
```

8 - Conditional Rendering – we'll use it later e.g. at Single Page Application

https://www.w3schools.com/REACT/react_conditional_rendering.asp
https://react.dev/learn/conditional-rendering

Your components will often need to display different things depending on different conditions. In React, you can conditionally render JSX using JavaScript syntax like if statements, &&, and ? : operators.

Hello React-1

Hello React-3

Hello React-5

Hello React-7

exercise-08.html

```
……..
    <script type="text/babel">
```

```
    function Conditional1() {
      const cond = true;
      if (cond)
        return <h1>Hello React-1</h1>;
      return <h1>Hello React-2</h1>;
    }
    ReactDOM.createRoot(document.getElementById('root1')).render(<Conditional1 />);

    function Conditional2() {
      const cond = true;
      return (
        <>
          {cond && <h1>Hello React-3</h1>}
          {!cond && <h1>Hello React-4</h1>}
        </ >
      );
    }
    ReactDOM.createRoot(document.getElementById('root2')).render(<Conditional2 />);

    function Conditional3() {
      const cond = true;
      return (
        <>
          { cond ? <h1>Hello React-5</h1> : <h1>Hello React-6</h1> }
        </ >
      );
    }
    ReactDOM.createRoot(document.getElementById('root3')).render(<Conditional3 />);

    function Conditional4() {
      const cond = true;
      let message;
      if (cond)
        message = <h1>Hello React-7</h1>;
      else
        message = <h1>Hello React-8</h1>;
      return message;
    }
    ReactDOM.createRoot(document.getElementById('root4')).render(<Conditional4 />);
  </script>
……
```

Don't change state while rendering! => otherwise there may be an infinite loop due to re-rendering!

exercise-09a.html

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>
```

```
  <body>
    <div id="root1"></div>
    <div id="root2"></div>
    <script type="text/babel">
      const initialValue = React.useState;
      function Example() {
        const [count, setCount] = initialValue(0);
        setCount(count+1);
        return <p>{count}</p>;
      }
      const root1 = ReactDOM.createRoot(document.getElementById('root1'));
      root1.render(<Example />);
    </script>
  </body>
</html>
```

At **setCount(count+1);** we increase the value of count. When the count changes, the component is re-rendered => we increase the value of count again => the component is re-rendered => …
infinite render

right click on the page / Inspect / Console
Uncaught Error: Too many re-renders. React limits the number of renders to prevent an infinite loop.

Do not change state while rendering! => otherwise there may be an infinite loop due to re-rendering!

If instead of: setCount(count+1);
we write: ()=>setCount(count+1);
this will not execute: ()=>setCount(count+1); because it is just a function definition that is not called anywhere.

Solution: useEffect

exercise-09b.html

```
<!DOCTYPE html>
<html>
  <head>
    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>
  <body>
    <div id="root1"></div>
    <div id="root2"></div>
    <script type="text/babel">
      const initialValue = React.useState;
      function Example() {
        const [count, setCount] = initialValue(0);
          React.useEffect(() => {
                setCount(c => c + 1); // VAGY: setCount(count + 1);
          }, []);
        return <p>{count}</p>;
      }
```

```
    const root1 = ReactDOM.createRoot(document.getElementById('root1'));
    root1.render(<Example />);
  </script>
 </body>
</html>
```

With useEffect it doesn't cause infinite rendering because **useEffect doesn't run during the render phase, but after rendering.**

Without dependency it would also be infinite rendering:

```
React.useEffect(() => {
        setCount(c => c + 1);
});
```

https://medium.com/@eric_abell/mastering-the-basics-of-reacts-useeffect-a-beginner-s-guide-3173c399b9e1

1. **Run on Every Render (No Dependency Array):**

```
useEffect(() => {
  console.log('Runs after every render');
});
```

2. **Run Only Once (Empty Dependency Array):**

```
useEffect(() => {
  console.log('Runs only once (like componentDidMount)');
}, []);
```

3. **Run When State/Props Change:**

```
useEffect(() => {
  console.log('Runs when count changes');
}, [count]);
```

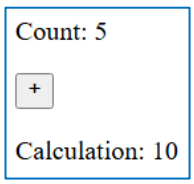
By passing [count] as a dependency, the effect only runs when the count state changes, making it more efficient.

9 – useEffect

https://react.dev/reference/react/useEffect
https://www.w3schools.com/react/react_useeffect.asp
https://www.geeksforgeeks.org/reactjs/reactjs-useeffect-hook/

The useEffect Hook allows you to perform side effects in your components.
Some examples of side effects are: fetching data, directly updating the DOM, and timers.
useEffect accepts two arguments. The second argument is optional:
useEffect(<function>, <dependency>)
Dependency array: React re-runs the effect if any of the values in this array change.

Exercise



Browser title: 

exercise-09c.html

```
<!DOCTYPE html>
<html>
 <head>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
     function Counter() {
        const [count, setCount] = React.useState(0);
        const [calculation, setCalculation] = React.useState(0);
// If the cound useState variable changes, it runs the useEffect function:
        React.useEffect(() => {
           setCalculation(() => count * 2);
           document.title = `You clicked ${count} times`;
        }, [count]);
        return (
           <>
              <p>Count: {count}</p>
              <button onClick={() => setCount((c) => c + 1)}>+</button>
              <p>Calculation: {calculation}</p>
           </>
        );
     }
     ReactDOM.createRoot(document.getElementById('root')).render(<Counter />);
  </script>
 </body>
</html>
```

The time of effect of setState, the time of effect of re-rendering, Batching, setState parameter can be a function, Updater Function

The time of effect of setState

Calling the Set function does not change the current state in the already executing code:

```
function handleClick() {
  setName('Robin');
  console.log(name); // Still "Taylor"!
}
```

It only affects what useState will return starting from the next render.
https://react.dev/reference/react/useState

The time of effect of re-rendering

React waits until all code in the event handlers has run before processing your state updates.
Therefore, the effect of setState is not visible immediately after the statement.
https://react.dev/learn/queueing-a-series-of-state-updates

Until re-rendering, the useState variable still contains the old value.

```
const [count, setCount] = useState(0);
const handleClick = () => {
   setCount(count + 1);        // setCount(0 + 1)
   setCount(count + 1);        // setCount(0 + 1)
   setCount(count + 1);        // setCount(0 + 1)
};
```

The value of count after handleClick will be 1 and not 3.

React waits for the codes inside the event handlers to finish before processing your state updates and then re-render. React calls it **batching**.
https://dev.to/vatana7/ever-wonder-why-reacts-setstate-doesnt-update-immediately-3nbc

This lets you update multiple state variables without triggering too many re-renders. But this also means that the UI won't be updated until after your event handler, and any code in it, completes. This behavior, also known as **batching**, makes your React app run much faster.
https://react.dev/learn/queueing-a-series-of-state-updates

If we want to change the state immediately => Updater Function: The parameter of setState is function

In the following example, the parameter of setState is a function: Updater Function.
So in order to immediately update the state before next render, you can React's Updater Function.

```
const [count, setCount] = useState(0);
const handleClick = () => {
   setCount(count + 1);        // React adds "replace with 1" to its queue.
   setCount((count) => count + 1); // updater function: React adds that function to queue.
};       // next render count = 2
```

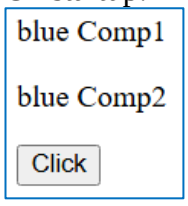
similarly:

```
function handleClick() {
  setAge(a => a + 1); // setAge(0 => 1)
  setAge(a => a + 1); // setAge(1 => 2)
  setAge(a => a + 1); // setAge(2 => 3)
}
```

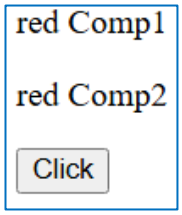
Here, a => a + 1 is your **updater function**. It takes the pending state and calculates the next state from it. React puts your updater functions in a queue. Then, during the next render, it will call them in the same order.
https://react.dev/reference/react/useState

10 - Changing the useState of the calling component from the called component

On startup:



After clicking the button:



exercise-10.html

```
<!doctype html>
<html lang="en">
 <head>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
 </head>
 <body>
  <div id="root"></div>
  <script type="text/babel">
     function Comp2(vars) {
        return (
           <>
              <p>{vars.color} Comp2</p>
              <button onClick={() => {vars.setCol("red");}}>Click</button>
           </>
        );
     }
     function Comp1() {
        const [color, setColor] = React.useState("blue");
        return (
           <>
              <p>{color} Comp1</p>
              <Comp2 color={color} setCol={setColor}/>
           </>
        );
     }
     ReactDOM.createRoot(document.getElementById('root')).render(<Comp1 />);
  </script>
 </body>
</html>
```

Using external React files => Web server is needed – Only informative text

We will store the components in external React files later.

A, The next exercise without web-server gives CORS error.

Comp1.js

```
class Class1 extends React.Component {
   render(){
      return ( <div>Hello From React </div> );
   }
}
const root = ReactDOM.createRoot(document.getElementById('root'));
```

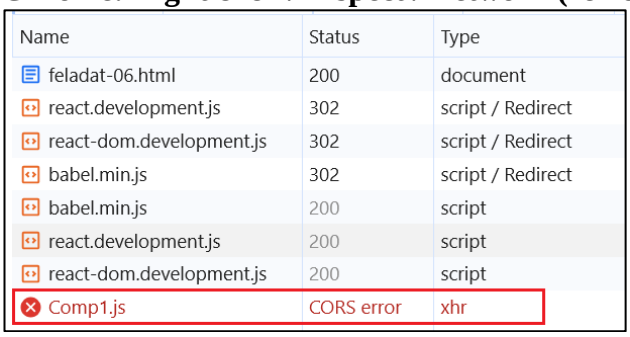
root.render(<Class1 />);

exercise-20.html

```
<!DOCTYPE html>
<html>
  <head>
……..
  </head>
  <body>
    <div id="root"></div>
    <script type="text/babel" src="Comp1.js"></script>
  </body>
</html>
```

Open **exercise-20.html** file in browser => Empty page

Chrome: Right click / Inspect / Network (refresh the page):



Name	Status	Type
feladat-06.html	200	document
react.development.js	302	script / Redirect
react-dom.development.js	302	script / Redirect
babel.min.js	302	script / Redirect
babel.min.js	200	script
react.development.js	200	script
react-dom.development.js	200	script
Comp1.js	CORS error	xhr

The browser has not allowed the code to run: for security reasons. Reason for blocking:
CORS: Cross-origin resource sharing
https://en.wikipedia.org/wiki/Cross-origin_resource_sharing

> Cross-origin resource sharing (CORS) is a mechanism to safely bypass the same-origin policy, that is, it allows a web page to access restricted resources from a server on a domain different than the domain that served the web page. A web page may freely embed cross-origin images, stylesheets, scripts, iframes, and videos. Certain "cross-domain" requests, notably Ajax requests, are forbidden by default by the same-origin security policy.

B, Copying the files to web-server there is no error.

- On local computer using XAMPP
  Try it!
  - XAMPP Control Panel: Start Apache web-server
  - copy files to c:\xampp\htdocs\exercise folder:
  - http://localhost/exercise/exercise-14.html
    Prints: Hello From React
- Using Internet host pl. nethely.hu.

Browsers limit features when loading apps from file:// (local disk) instead of http:// or https:// due to security, mainly the Same-Origin Policy

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS
https://webmasters.stackexchange.com/questions/124869/what-are-the-benefits-of-utilizing-a-web-server-vs-opening-an-html-file-directly

blocking cross-origin requests (like API calls) and preventing full functionality like database access or server-side features unless you use a proper web server (even a simple local one like npx http-server) to mimic a real web environment.

Why this happens:

- **Same-Origin Policy (SOP):** Browsers restrict scripts from one origin (e.g., http://localhost:3000) from accessing data from another (e.g., https://api.example.com) for security.
- **CORS Restrictions:** When you open a local file, it has no origin, so browsers block fetches to other domains, even your own local backend, without proper Cross-Origin Resource Sharing (CORS) headers, which only a server provides.
- **Server-Side Tech:** Features like databases, user authentication, or complex backend logic (PHP, Python, Node.js) require a server to run; they can't execute from a local file.

Because of the above: from now on, we will use web server for all React exercises

5a – React – Development without Babel/standalone – Install React on local computer - vite + React - we can create a faster application this way: no need to compile at runtime – We create the app, then compile it (Build), then run it

Vite is a build tool that aims to provide a faster and leaner development experience for modern web projects. It provides features to package and runs source code, provides a development server for local development and a build command to deploy your app to a production server.

Install Node.js:

Webpage and download: https://nodejs.org
Download the LTS (Long Term Support) version https://nodejs.org/en/download/
Download Windows Installer, and install it

Install with Vite – Only informative text

Only informative text: I installed and copied to **Installed-React-*.*.*.zip** file.

Install with create-react-app – Old method – we don't use it

https://react.dev/blog/2025/02/14/sunsetting-create-react-app
Today, we're deprecating Create React App for new apps, and encouraging existing apps to migrate to a framework, or to migrate to a build tool like Vite, Parcel, or RSBuild.

https://www.w3schools.com/react/react_getstarted.asp

Install Vite + React together

npm create vite@latest react-app -- --template react
cd react-app
npm install

Start with vite server

npm run dev
http://localhost:5173/

Little sample application created with installation

The files

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

/src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

StrictMode

https://react.dev/reference/react/StrictMode

<StrictMode> lets you find common bugs in your components early during development.

App.jsx

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)
  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
        <p>
          Edit <code>src/App.jsx</code> and save to test HMR
        </p>
      </div>
      <p className="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}
export default App
```

Build – creating the runnable (without Vite) app => You need a web server to run it (otherwise, CORS error!)

npm run build **=> It creates the dist folder**
with the next files:
index.html
vite.svg
assets/
index-***.css
index-***.js
react-***.svg

Open dist/index.html file => empty page
right click on page / Inspect / Console:
Access to script at 'file:///C:/assets/index-SguGrCfz.js' from origin 'null' has been blocked by CORS policy
CORS: Cross-origin resource sharing
We've already studied the CORS error: we studied that a server is required to run it

Running the built application (without Vite) – with a web server

Running on a local computer – e.g. with XAMPP – We will also use it for the backend (PHP) exercise

Start the Apache web server in XAMPP

The dist folder contains the executable application. **Where do we copy it?**

e.g. it is not good to copy it to the **c:\xappp\htdocs\exercise** folder, because this is in dist/index.html:

```
……..
   <script type="module" crossorigin src="/assets/index-***.js"></script>
   <link rel="stylesheet" crossorigin href="/assets/index-***.css">
…….
```

The app is looking for the /assets folder here: **c:\xappp\htdocs**

because **c:\xappp\htdocs** is set as the working folder in XAMPP by default.

=> C:\xampp\apache\conf\httpd.conf file

right click on the page in the browser => Inspect => Network => Refresh page

Not found: http://localhost/assets/.....

You can set the default folder in the vite.config.js file:

```
……..
export default defineConfig({
  base: '/exercise/',
  plugins: [react()],
})
……..
```

buid again: **npm run build**

the dist/index.html file already contains these:

```
……..
   <script type="module" crossorigin src="/exercise/assets/index-***.js"></script>
   <link rel="stylesheet" crossorigin href="/exercise/assets/index-***.css">
…….
```

Copy the contents of the dist folder to the **c:\xampp\htdocs\exercise** folder.

http://localhost/exercise/
It works well

Running on an Internet server – Required for homework

Register with a free hosting provider, e.g. infinityfree.com, nethely.hu, …
Description for it:
InfinityFree-Free-Internet-Hosting-Provider.docx,
Internet-Hosting-Provider-PHP-Hungarian Nethely.hu-in English.docx

In the vite.config.js file:

a, the base folder should not be specified (by default: base: '/'):

…….
export default defineConfig({
  plugins: [react()],
})
……..

OR:
…….
export default defineConfig({
  base: '/',
  plugins: [react()],
})
……..

build again: **npm run build**

You cab find this code in dist/index.html file:
……..
    <script type="module" crossorigin src="/assets/index-***.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-***.css">
…….

Copy the contents of the **dist** folder to the remote server using FTP.

Enter the URL in your browser, set on the hosting:
It works!

Run on Internet server - Needed for Homework

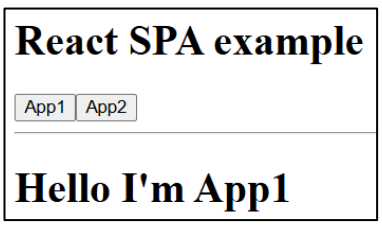
Register with a free hosting provider e.g. nethely.hu, infinityfree.com

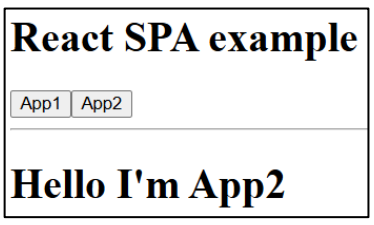
Description:
Internet-Hosting-Provider-PHP-Hungarian Nethely.hu-in English.docx
InfinityFree-Free-Internet-Hosting-Provider.docx

Copy the contents of the **dist** folder to the remote server using FTP.
Enter the URL, set on the hosting in your browser:
It works!

01A- Single-page Application (SPA)

Clicking the first button renders the App1 application, clicking the second button renders App2.
It loads App1 on startup.





index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.jsx

```
import { useState } from 'react'
import App1 from "./App1";
import App2 from "./App2";

function App() {
  const [menu, setMenu] = useState("app1");
  return (
    <div>
      <h1>React SPA example</h1>
      <nav>
        <button onClick={() => setMenu("app1")}>App1</button>
        <button onClick={() => setMenu("app2")}>App2</button>
      </nav>
      <hr />
// Conditional rendering, we've studied about it:
//         If menu === "app1", then insert App1:
//         OR: { menu === "app1" ? <App1 /> : <App2 /> }
      {menu === "app1" && <App1 />}
      {menu === "app2" && <App2 />}
    </div>
  );
}
export default App
```

src/App1.jsx

```
function App1() {
```

```
  return (
    <div>
      <h1>Hello I'm App1</h1>
    </div>
  );
}
export default App1
```

src/App2.jsx

```
function App2() {
    return (
      <div>
        <h1>Hello I'm App2</h1>
      </div>
    );
  }
  export default App2
```

Run on local computer

npm run dev

http://localhost:5173/

Building and Run on remote server

npm run build

Copy the contents of the **dist** folder to the remote server using FTP.
Enter the URL set on the hosting in your browser:
It works well!

01B- Single-page Application (SPA) with Router – The previous SPA is simpler – Only informative text

The previous SPA is simpler:

- no need to install the Router module
- no need for the .htaccess file either

https://developer.mozilla.org/en-US/docs/Glossary/SPA
https://en.wikipedia.org/wiki/Single-page_application
https://www.bloomreach.com/en/blog/what-is-a-single-page-application

A single-page application (SPA) is a web app that is presented to the user through a single HTML page.

https://www.w3schools.com/REACT/react_router.asp

Install the react-router-dom modult: In Command prompt:
npm i -D react-router-dom@latest
If errors are printed:
npm audit fix --force

npm create vite@latest router -- --template react
cd router
npm install
Copy the files from **Solutions.zip**.
npm run dev
http://localhost:5173/

Start on local computer

npm run dev

http://localhost:5173/

Build and Run on a Remote Server – If using a Router, you need a .htaccess file!

npm run build

Copy the contents of the **dist** folder to the remote server using FTP.
Enter the URL, set on the hosting site in your browser:
It seems to work fine!

Problem
We go to a page, e.g. Contact
/contact
In the browser: Reload => gives a 404 error
Cause: the server is trying to load the /contact path, but it does not exist on the server

Solution
Send all calls to the index.html file:

.htaccess

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /
  RewriteRule ^index\.html$ - [L]
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteCond %{REQUEST_FILENAME} !-l
  RewriteRule . /index.html [L]
</IfModule>
```

REACT sample applications on the web

https://www.codewithfaraz.com/article/196/25-react-projects-for-beginners-with-source-code
https://www.geeksforgeeks.org/reactjs-projects/
https://legacy.reactjs.org/community/examples.html

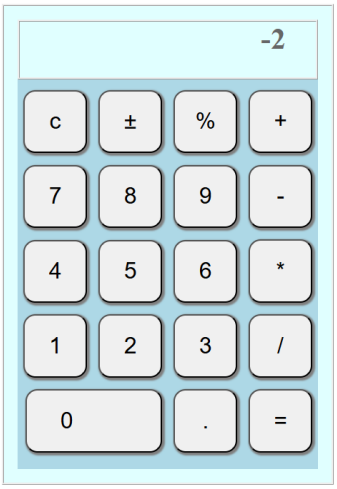
https://sean-smith.surge.sh/assets/portfolio/25-react-projects/index.html
https://www.freecodecamp.org/news/react-projects-for-beginners-easy-ideas-with-code/
https://codegnan.com/react-js-projects/
https://medium.com/design-bootcamp/top-5-react-projects-for-beginners-509c29c91336
https://www.zegocloud.com/blog/react-projects
https://devaradise.com/react-example-projects/
https://hackr.io/blog/react-projects

https://github.com/ianshulx/React-projects-for-beginners

https://www.freecodecamp.org/news/master-react-by-building-25-projects/
https://contactmentor.com/best-react-projects-for-beginners-easy/

5b – React sample exercises 5-5 minutes – We don't study the code, we just try the applications

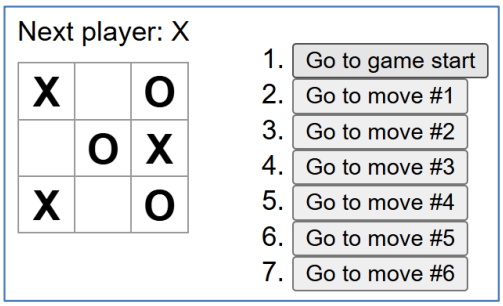
1-Calculator



Copy files from **Solutions.zip** to the extracted **Installed-React-*.*.*.zip** application.

npm run dev
http://localhost:5173/

2-Tic-Tac-Toe



Copy files from **Solutions.zip** to the extracted **Installed-React-*.*.*.zip** application.

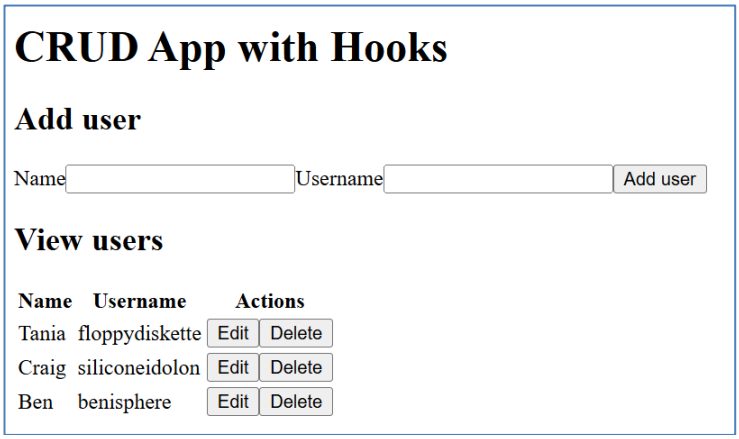
npm run dev
http://localhost:5173/

6 - React-CRUD – Single Page application (SPA) - The application restarts when the page is refreshed => Axios will solve it

CRUD: Create, Read, Update, Delete

Single-page application (SPA): e.g. for the Create and Update operations, we add different components to the page (AddUserForm, EditUserForm).

Based:
https://stackblitz.com/edit/simple-react-crud
https://codesandbox.io/p/sandbox/crud-app-react-hooks-76w96



Copy files from **Solutions.zip** to the extracted **Installed-React-*.*.*.zip** application.

npm run dev
http://localhost:5173/

index.html
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>react-app</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/main.jsx
```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
```

```
    <App />
  </StrictMode>,
)
```

The role of App.jsx is to:

- store the most important, global useState variables. We will modify these from subcomponents
- store functions to modify global useState variables
  useState variables have a Set function, but we also do other things with them, e.g. set the value of the editing variable. e.g.:

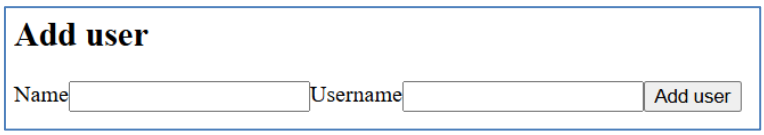
```
const editRow = user => {
        setEditing(true);
        setCurrentUser(user);
};
```

  We will pass these functions to the subcomponents and then make the modification from there. e.g.:

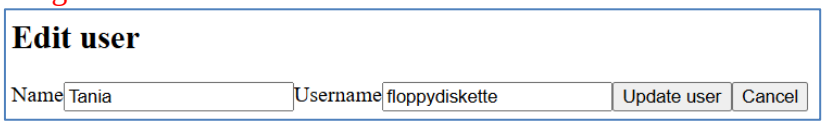
  tables/UserTable.jsx: props.editRow(user), props.deleteUser(user.id)
  forms/ AddUserForm.jsx: props.setCurrentUser(initialFormState), props.addUser(user);
- display the page elements

src/App.jsx

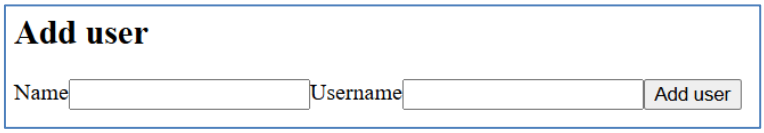
```
import React, { useState } from "react";
// Components:
import UserTable from "/src/tables/UserTable";
import EditUserForm from "/src/forms/EditUserForm";
import AddUserForm from "/src/forms/AddUserForm";

const App = () => {
  const usersData = [
    { id: 1, name: "Tania", username: "floppydiskette" },
    { id: 2, name: "Craig", username: "siliconeidolon" },
    { id: 3, name: "Ben", username: "benisphere" }
  ];

// Setting useState variables:
// We store the users in the users (useState) array:
  const [users, setUsers] = useState(usersData);

// We store the current user in the currenUser (useState) array:
//        It has to be stored here and not in the EditUserForm because:
//                When we click the Edit button on the table, it calls the editRow function:
//                        props.editRow(user);
//                which loads the selected user into the currentUser variable:
//                        setCurrentUser(user);
//        We read it from here in the EditUserForm as well
  const [currentUser, setCurrentUser] = useState("");

// Initially we add the AddUser section tot he page.
//        After the operations, return here: setEditing(false);
//        It inserts the AddUserForm above the table:
```



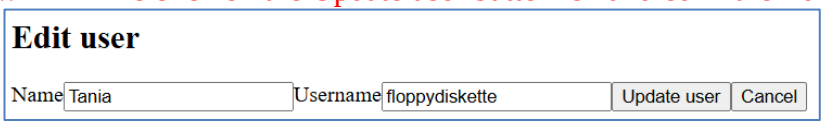
```
// for editing the EditUser section:
```




```
// The editing useState variable indicates whether we are creating a new user (AddUser),
//        or modifying an existing user (EditUser). true: Edit, false: Add
  const [editing, setEditing] = useState(false);

// Adds the user specified in the parameter to the users array.
//        user: parameter of the addUser function
//        call it in AddUserForm.jsx: props.addUser(user);
  const addUser = user => {
// Sets the user id:
    user.id = users.length + 1;
// [...users, user]: adds the user element to the users array
//        https://www.geeksforgeeks.org/javascript/add-elements-to-a-javascript-array
    setUsers([...users, user]);
  };

// We call it in UserTable: props.deleteUser(user.id)
// Deletes the user with the given id from the users array:
// id: parameter:
  const deleteUser = id => {
// It returns to this default state:
//        It inserts the AddUserForm above the table:
```




```
//
    setEditing(false);
// Deletes the user with the given id from the users array:
//        filter:  https://www.w3schools.com/jsref/jsref_filter.asp
//                 creates a new array filled with elements that pass a test provided by a function.
//                 does not change the original array.
//                 result: An array of elements that pass the test.
    setUsers(users.filter(user => user.id !== id));
  };

// We call it in UserTable: props.editRow(user);
// user: parameter:
  const editRow = user => {
// This status is set:
//        It inserts the EditUserForm above the table:
//        The click on the Update user button is handled in the EditUserForm.
```




```
    setEditing(true);
// Sets the user specified in the parameter as the current user:
    setCurrentUser(user);
  };

// It changes the user with the given id to the new user
```

```
//        We call it in EditUserForm: props.updateUser(user.id, user);
//        It has 2 parameters: user.id, user
  const updateUser = (id, updatedUser) => {
// It returns to this default state:
//        It inserts the AddUserForm above the table:
```


```
//
    setEditing(false);
// It changes the user with the given id to the new user
//        users.map: https://www.w3schools.com/jsref/jsref_map.asp
//                creates a new array from calling a function for every array element.
//                does not change the original array.
//                For each element of the users array: if user.id is equal to id, then the given element will be
//                        updatedUser, otherwise the original user.
    setUsers(users.map(user => (user.id === id ? updatedUser : user)));
  };

  return (
    <div>
      <h1>CRUD App with Hooks</h1>
      <div>
        <div>
          <div>
// If editing=true then "Edit user" otherwise "Add user"
            <h2>{editing ? "Edit user" : "Add user"}</h2>
// If editing=false then
//        Places the AddUserForm component with the parameter. We see the role of parameter
//                in the AddUserForm.
//                We studied that we call a component in the following form: <Car color="red"/>
            {!editing ? (
              <AddUserForm
                addUser={addUser}
              />
// If editing=true then Places the EditUserForm component with the parameters.
            ):(
              <EditUserForm
                setEditing={setEditing}
                currentUser={currentUser}
                setCurrentUser={setCurrentUser}
                updateUser={updateUser}
              />
            )}
          </div>
        </div>
        <div>
          <h2>View users</h2>
// Places the UserTable component with the parameters.
          <UserTable users={users} editRow={editRow} deleteUser={deleteUser} />
        </div>
      </div>
    </div>
```

```
  );
};
export default App;


// UserTable creates the table:
```

Name	Username	Actions	
Tania	floppydiskette	Edit	Delete
Craig	siliconeidolon	Edit	Delete
Ben	benisphere	Edit	Delete

```
//
```

tables/UserTable.jsx

```
import React from "react";
const UserTable = props => (
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Username</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      {props.users.length > 0 ? (
// Loops through all users:
        props.users.map(user => (
          <tr key={user.id}>
            <td>{user.name}</td>
            <td>{user.username}</td>
            <td>
              <button onClick={() => {props.editRow(user);}}>Edit</button>
              <button onClick={() => props.deleteUser(user.id)}>Delete</button>
            </td>
          </tr>
        ))
      ) : (
        <tr>
          <td colSpan={3}>No users</td>
        </tr>
      )}
    </tbody>
  </table>
);
export default UserTable;

// We modify the useState variable of the calling component (App.jsx) in several cases. e.g.:
//         props.setEditing(false);                    props.setCurrentUser(initialFormState);
//         props.updateUser(user.id, user)         props.addUser(user)
// AddUserForm creates the form above the table:
```

Name Username Add user

```
//
```

forms/AddUserForm.jsx

```
import React, { useState} from "react";
```

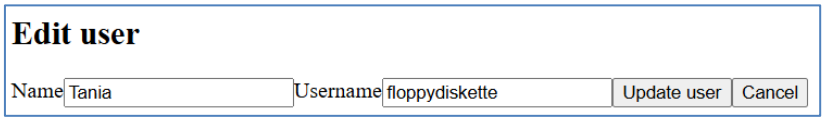
```
const AddUserForm = props => {
// useState variable to store user
//        Read the value of the input field from here: value={user.name}
  const [user, setUser] = useState({name:"",username:""});

// If either of the two input fields is modified, this function is called:
//        It modifies the value of the user useState variable that belongs to the given input field.
  const handleInputChange = event => {
// read the name and value of the given input field:
    const { name, value } = event.target;
// { ...user, [name]: value }: creates a new object
//        [name]: the value of the variable name
//        The [name]: value: adds or modifies a field to the user array
//        If the key already exists, the new value is overwritten
// Modifies the user array: the value variable will be the value of the name key:
    setUser({ ...user, [name]: value });
  };

  return (
    <form
      onSubmit={event => {
        event.preventDefault();
// If we don't give one of the two input fields, the function exits:
        if (!user.name || !user.username) return;
// Add the new user to the users array:
        props.addUser(user);
// Clears the input fields:
        setUser({name:"",username:""});
      }}
    >
      <label>Name</label>
// First input field:
      <input type="text" name="name" value={user.name} onChange={handleInputChange}/>
      <label>Username</label>
// Second input field:
      <input type="text" name="username" value={user.username} onChange={handleInputChange}/>
      <button>Add user</button>
    </form>
  );
};
export default AddUserForm;
```

There are many similarities to the previous component:

forms/EditUserForm.jsx

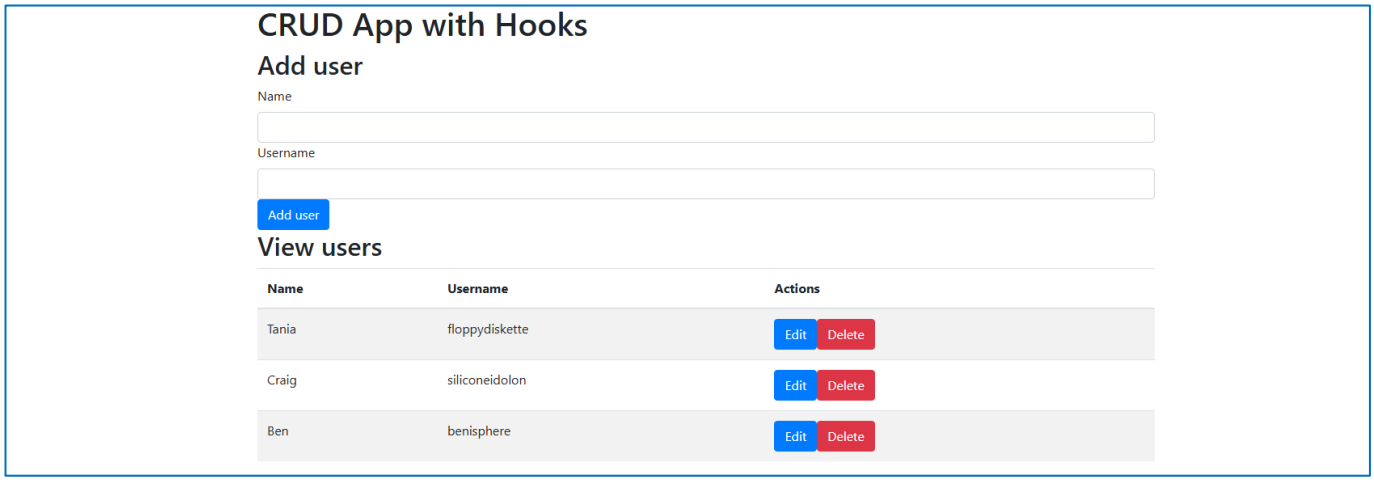
```
import React, { useState, useEffect } from "react";

const EditUserForm = props => {
// the selected user:
  const [user, setUser] = useState(props.currentUser);

  const handleInputChange = event => {
```

```
    const { name, value } = event.target;
    setUser({ ...user, [name]: value });
  };

// without this useEffect: Clicking the Edit button loads the EditUserForm above the table, loading
// the selected user's data:
```




```
//
//        But if we then click another Edit button, it doesn't fill the form with this user's
//        data, but the previous data remains.
//                When we first click the Edit button, the user variable gets its initial value:
//                        const [user, setUser] = useState(props.currentUser);
//                The useState(props.currentUser); only sets the initial value, so when we click the Edit
//                button again, the user variable will not change, so the input fields
//                value will not change. To change it, we need to use useEffect:
//                it modifies the user variable when the props change:
  useEffect(() => {
    setUser(props.currentUser);
  }, [props]);
// We studied that state variables cannot be changed during rendering, because it results in infinite
//        rendering, so it would not be good to just write this instead of the previous one:
//        setUser(props.currentUser);
  return (
    <form
      onSubmit={event => {
        event.preventDefault();
// Changes the User with the given id to user:
        props.updateUser(user.id, user);
      }}
    >
      <label>Name</label>
      <input type="text" name="name" value={user.name} onChange={handleInputChange}/>
      <label>Username</label>
      <input type="text" name="username" value={user.username} onChange={handleInputChange}/>
      <button>Update user</button>
      <button onClick={() => props.setEditing(false)}>Cancel</button>
    </form>
  );
};
export default EditUserForm;
```

React CRUD – 2 – with Bootstrap formatting - 5 mins



=> **Solutions.zip**

7 - JavaScript-Ajax-Fetch API-CRUD - Save data to the server (database) - We modify the page without reloading it

- We have already created a CRUD application with JavaScript, but it does not save the data to the server. Here, we save the data to the database on the server.
- In Seminar, we also create a CRUD application with PHP, but there we reload the page for each operation. Here, we modify the website with JavaScript without refreshing the page.

CRUD: Create, Read, Update, Delete

Here we use a server-side language (PHP) and a database on the server side. These were learned in practice. Here we do not relearn that knowledge, but we use it.

Introduction – Only informative text

AJAX: Asynchronous JavaScript And XML.

https://www.w3schools.com/js/js_ajax_intro.asp
https://en.wikipedia.org/wiki/Ajax_(programming)

AJAX is a developer's dream, because you can:

- Read data from a web server - after the page has loaded
- Update a web page without reloading the page
- Send data to a web server - in the background

We don't reference the PHP file in the HTML file: we will only do so in the Javascript file!

- There is no direct connection between HTML and PHP at all
  JS communicates with both HTML and PHP separately
- **JS acts as a controller** (after the page loads)
  - JavaScript is playing an increasingly important role these days.

JSON: JavaScript Object Notation

We will see that when calling AJAX and Fetch API, the data is transferred in JSON format {…}.
We do not need to know the JSON format because the protocol solves it in the background, so the following is Only informative text:

JSON (JavaScript Object Notation) is a lightweight, text-based, language-independent data-interchange format. It is easy for humans to read and write and for machines to parse and generate. Widely used in web applications, it structures data using key-value pairs (e.g., "key": "value") and arrays.
Key Characteristics and Usage:
Structure: Built on two main structures: A collection of name/value pairs (objects) and an ordered list of values (arrays).
Data Types: Supports strings, numbers, booleans, null, arrays, and objects.
Common Use: Primarily used to transmit data between a server and a web application.
Syntax Rules: Keys must be in double quotes; data is comma-separated.

Language Independence: Although derived from JavaScript, JSON is supported by most modern programming languages (Python, Java, C++, etc.).

A **JSON** egy kis méretű, **szöveg alapú szabvány**, ember által olvasható **adatcserére**. A JavaScript szkriptnyelvből alakult ki egyszerű adatstruktúrák és asszociatív tömbök reprezentálására (a JSON-ban objektum a nevük). A JavaScripttel való kapcsolata ellenére **nyelvfüggetlen**, több nyelvhez is van értelmezője.
A JSON-t **legtöbbször egy szerver és egy kliens számítógép közti adatátvitelre** használják (legtöbbször AJAX technológiával), **az XML egyik alternatívájaként**. Általánosságban **strukturált adatok tárolására, továbbítására szolgál**.

Example:

```
{
  "name": "John",
  "age": 30,
  "isStudent": false,
  "hobbies": ["reading", "coding"]
}
```

AJAX vs. Fetch API – AJAX is the older, Fetch API is the newer technology

but there are places where Fetch API is also called AJAX.

AJAX
Uses the older XMLHttpRequest object for requests.
Relies on callback functions to handle responses.

Fetch API
Uses the modern fetch() function based on Promises.
Uses Promises, enabling .then(), .catch(), and async/await.

The Fetch API is a modern alternative to AJAX with a cleaner syntax and support for Promises

https://www.geeksforgeeks.org/javascript/difference-between-ajax-and-fetch-api/
https://lemon.io/answers/ajax/is-ajax-better-than-the-fetch-api

Exercise – 1 – with GET, POST, PUT, DELETE methods – Design with AI

Design with AI - Using artificial intelligence in web development

ChatGPT: Create a CRUD application with HTML, CSS, JavaScript, PHP, MySQL, Fetch API. For Fetch API use GET, POST, PUT, DELETE methods. In PHP use the PDO class for the database. Use Bootstrap for formatting.

The four CRUD operations (Create, Read, Update, Delete) can be handled on the server side by querying (GET, POST, PUT, DELETE) from PHP $_SERVER["REQUEST_METHOD"].
https://www.php.net/manual/en/reserved.variables.server.php
You don't need to use routes for this.
You can send GET, POST, PUT, DELETE requests with JavaScript, e.g. with the Fetch API.

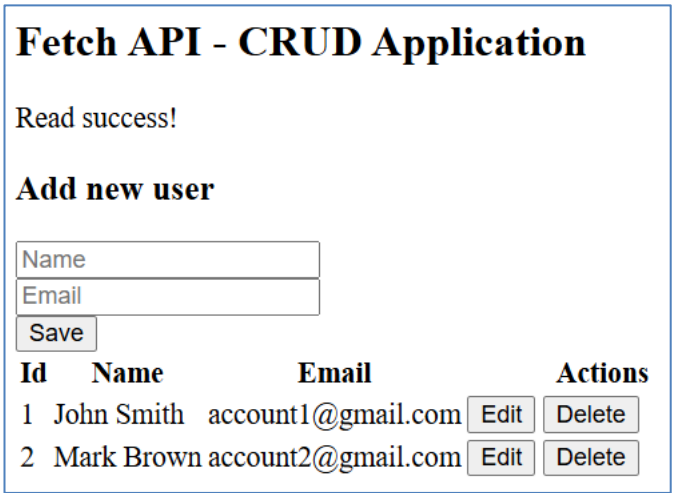
In the Seminar, we also create a CRUD application with PHP and HTML. Since we can only send requests with GET and POST methods with HTML, we use routes there and read the four operations (Create, Read, Update, Delete) from $_SERVER['QUERY_STRING'];

Fetch API:

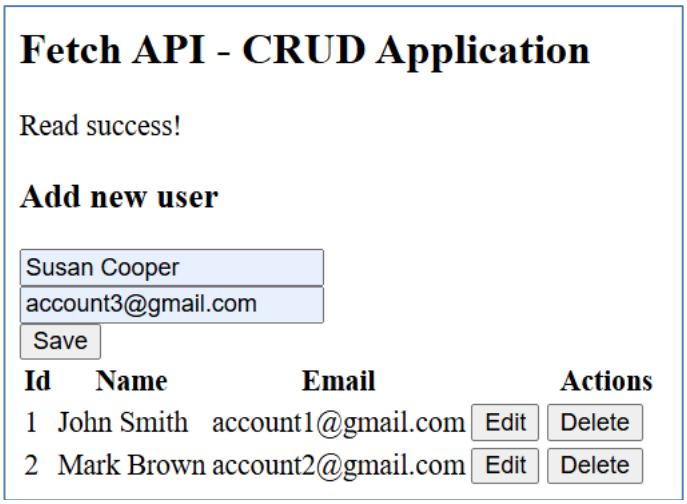
https://www.w3schools.com/jsref/api_fetch.asp

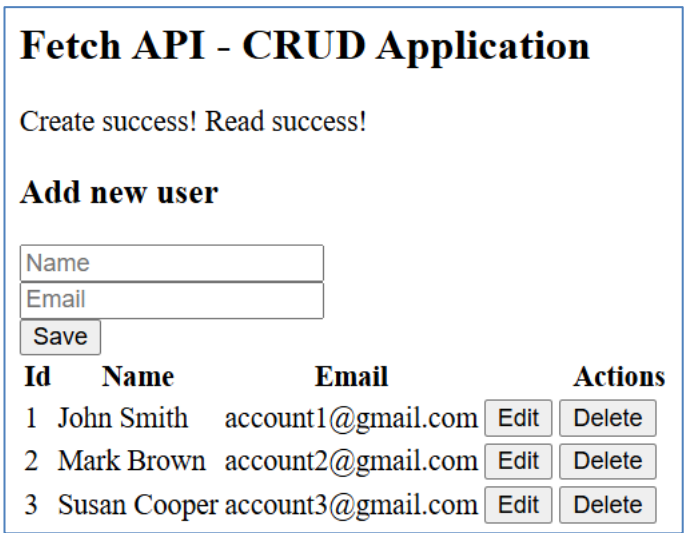
The fetch() method starts the process of fetching a resource from a server.

Main page:

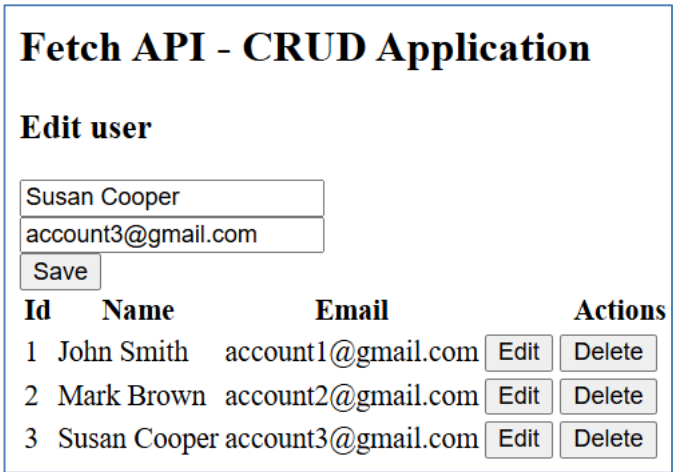


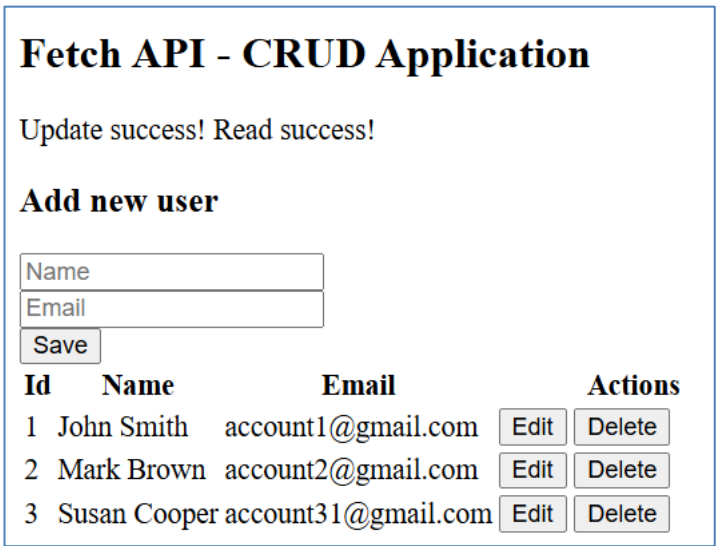
Create



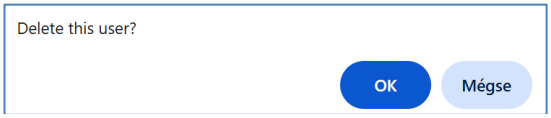


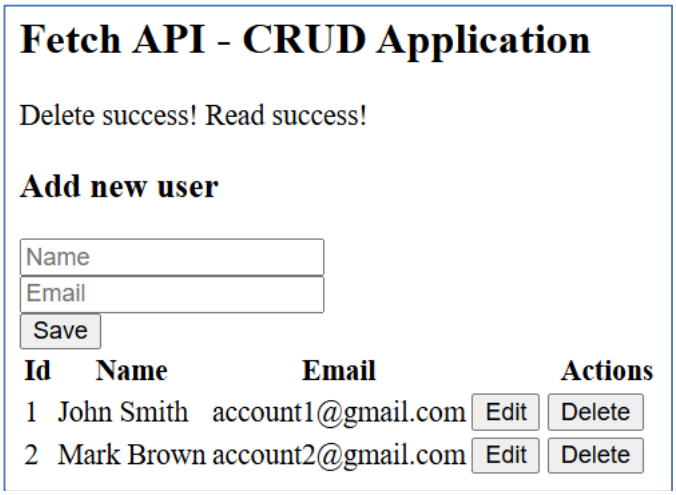
Edit





Delete:





1-Solution

It is a simple but complete AJAX CRUD app using:

HTML + Bootstrap (UI)
JavaScript + Fetch API (AJAX)
PHP (PDO) (backend)
MySQL (database)

Features Implemented

✓ Create
✓ Read
✓ Update
✓ Delete
✓ PDO + Prepared Statements
✓ Bootstrap UI

Folder Structure

c:/xampp/htdocs/fetch-crud/

```
├── index.html
├── script.js
├── api.php
└── db.php
```

http://localhost/fetch-crud/

HTML

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Fetch API CRUD</title>
</head>
<body>
   <div>
      <h2>Fetch API - CRUD Application</h2>
// To display messages for operations:
      <p id="message"></p>
// for displaying Add new user / Edit user:
      <h3 id="addedit"></h3>
// Form for Create and Edit operations
      <form id="userForm">
// At Edit, we store the ID of the user to be modified in this hidden field.
//         We will read it from here when saving
         <input type="hidden" id="id">
         <div>
            <div>
               <input type="text" id="name" placeholder="Name" required>
            </div>
            <div>
               <input type="email" id="email" placeholder="Email" required>
```

```
            </div>
            <div>
// We handle the button click in JavaScript:
//         document.getElementById("userForm").addEventListener("submit", saveUser);
                <button>Save</button>
            </div>
        </div>
    </form>
    <table>
        <thead>
            <tr>
                <th>Id</th>
                <th>Name</th>
                <th>Email</th>
                <th width="150">Actions</th>
            </tr>
        </thead>
        <tbody id="userTable"></tbody>
    </table>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Database (MySQL)

This example manages a Users table (id, name, email).

```
CREATE DATABASE crud;
USE crud;
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL
);
INSERT INTO users (name, email) VALUES
('John Smith', 'account1@gmail.com'),
('Mark Brown', 'account2@gmail.com');
```

Database Connection

db.php

```
<?php
$host = "localhost";
$db   = "crud";
$user = "root";
$pass = "";
try {
  $pdo = new PDO("mysql:host=$host;dbname=$db;charset=UTF8",$user,$pass,
  [PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION,PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC]);
```

```
} catch (PDOException $e) {
   die("Database connection failed");
}
```

PHP API

This single file handles **Create, Read, Update, Delete** via action.

api.php

```
<?php
header("Content-Type: application/json");
require "db.php";
// Read the sending method from the $_SERVER array: GET, POST, PUT, DELETE
//        https://www.php.net/manual/en/reserved.variables.server.php
$method = $_SERVER['REQUEST_METHOD'];
switch ($method) {
   case 'GET':
      try {
// Retrieve the users table data from the database and return it to the caller:
         $stmt = $pdo->query("SELECT * FROM users");
         $readData=$stmt->fetchAll();
// Transfers the data in a JSON object:
         echo json_encode(['status' => 'Read success!', "readData"=>$readData]);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Read error!']);
      }
      break;
   case 'POST':
      try {
Converts data sent in JSON to an array:
         $data = json_decode(file_get_contents("php://input"), true);
// Add a new record to the table with the given name and email data:
//        Prepared query: You learned about it in Seminar.
//        The ID is automatically assigned a value (AUTO_INCREMENT)
         $stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (?, ?)");
         $stmt->execute([$data['name'], $data['email']]);
         echo json_encode(['status' => 'Create success!']);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Create error!']);
      }
      break;
   case 'PUT':
      try {
         $data = json_decode(file_get_contents("php://input"), true);
// Modify the data of the record with the given ID:
         $stmt = $pdo->prepare("UPDATE users SET name=?, email=? WHERE id=?");
         $stmt->execute([$data['name'], $data['email'], $data['id']]);
         echo json_encode(['status' => 'Update success!']);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Update error!']);
      }
```

```
        break;
    case 'DELETE':
        try {
            $data = json_decode(file_get_contents("php://input"), true);
// Deletes the record with the given ID:
            $stmt = $pdo->prepare("DELETE FROM users WHERE id=?");
            $stmt->execute([$data['id']]);
            echo json_encode(['status' => 'Delete success!']);
        }
        catch(PDOException $e) {
          echo json_encode(['status' => 'Delete error!']);
        }
        break;
}
```

JavaScript

script.js

```
const api = "api.php";
// Call the saveUser function when the button is clicked on the form:
document.getElementById("userForm").addEventListener("submit", saveUser);
// If the page loads, call the fetchUsers() function:
window.onload = function() {
    fetchUsers();
};
function fetchUsers() {
// Changes the text of the addedit element:
    document.getElementById("addedit").innerHTML = "Add new user";
// The fetch() method starts the process of fetching a resource from the server.
//          https://www.w3schools.com/jsref/api_fetch.asp
//          Sends a POST request to the api.php script:
// If no method is specified, the request is sent using the GET method:
    fetch(api)
// If the server response is received:
// The server sends the data in JSON format (json_encode). We first convert this to an array:
        .then(res => res.json())
// We receive the data sent by the server in the data variable:
        .then(data => {
// Prints: Read success! / Read error!
            document.getElementById("message").innerText += " "+data.status;
// We create the HTML table in a string:
            let rows = "";
// We loop through each user:
            data.readData.forEach(user => {
                rows += `
                <tr>
                    <td>${user.id}</td>
                    <td>${user.name}</td>
                    <td>${user.email}</td>
                    <td>
// Clicking the Edit button calls the editUser function with the user parameter
//          We first convert this to JSON:
                        <button onclick='editUser(${JSON.stringify(user)})'>Edit</button>
```

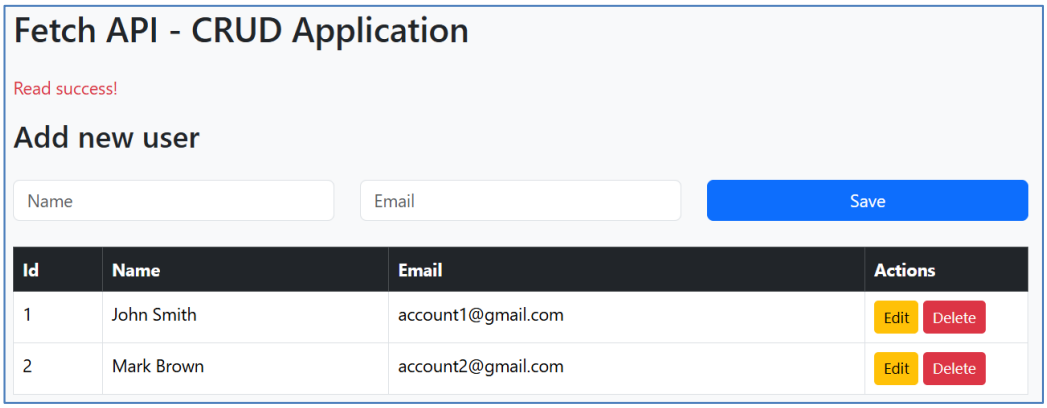
```
                    <button onclick='deleteUser(${user.id})'>Delete</button>
                </td>
            </tr>`;
        });
        document.getElementById("userTable").innerHTML = rows;
    });
}
// After clicking the button on the top form
//      This handles the Create and Update functions:
function saveUser(e) {
// Prevents the default button click event handler from running:
    e.preventDefault();
    const id = document.getElementById("id").value;
    const name = document.getElementById("name").value;
    const email = document.getElementById("email").value;
// We send this data with POST and PUT requests:
// The id is not used with POST requests
    const data = {
        id:id,
        name: name,
        email: email
    };
// If the ID is not an empty string, then PUT, otherwise POST is the method value:
    const $method = id ? "PUT" : "POST";
// Sends the request to the server using the given method:
    fetch(api, {
        method: $method,
        headers: {"Content-Type": "application/json"},
// Sends the data in the body. Converts it to JSON before sending:
        body: JSON.stringify(data)
    })
// If the response arrives from the server:
    .then(res => res.json())
    .then(data => {
// Clears the text fields:
        e.target.reset();
// Prints the operation message:
        document.getElementById("message").innerText = data.status;
// resets the ID value:
        document.getElementById("id").value = "";
        fetchUsers();
    });
}

// When the Edit button is clicked, this function is run:
function editUser(user) {
    document.getElementById("message").innerText = "";
    document.getElementById("addedit").innerHTML = "Edit user";
// Inserts the user's data into the form:
    document.getElementById("id").value = user.id;
    document.getElementById("name").value = user.name;
    document.getElementById("email").value = user.email;
}
```

```
// When the Delete button is clicked, this function is run:
function deleteUser(id) {
// In the Confirm window it asks for confirmation to delete:
    if (!confirm("Delete this user?")) return;
// Similar to the previous ones:
    fetch(api, {
        method: "DELETE",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({id})
    })
    .then(res => res.json())
    .then(data => {
        document.getElementById("message").innerText = data.status;
        fetchUsers();
    });
}
```

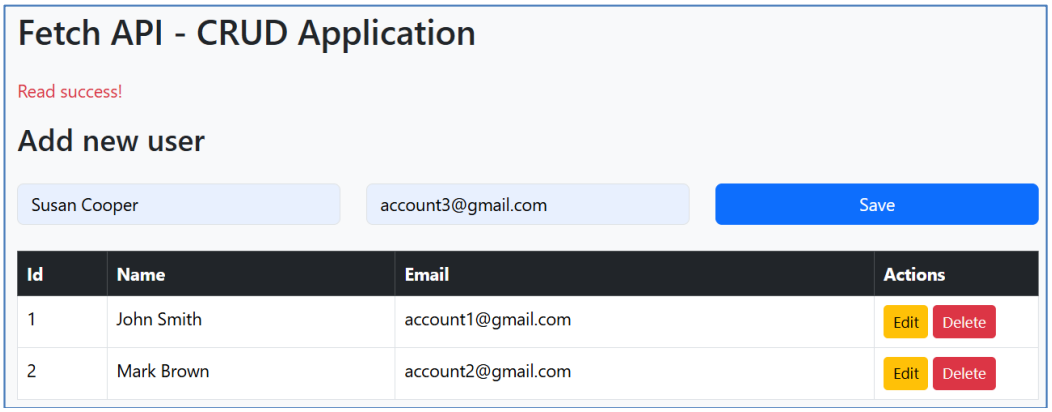
2 – Formatting with Bootstrap – 5 mins

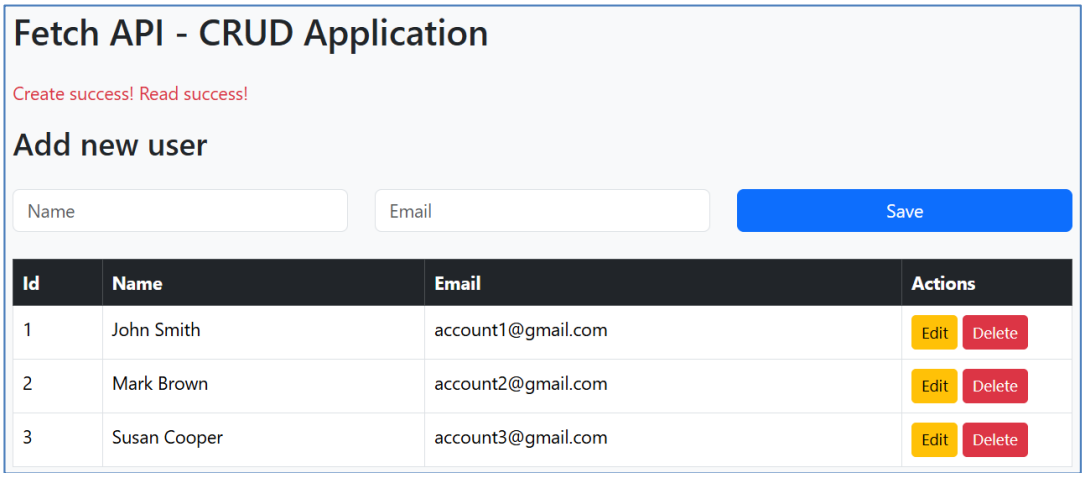
As the previous solution, we only format it with Bootstrap:

Main page:

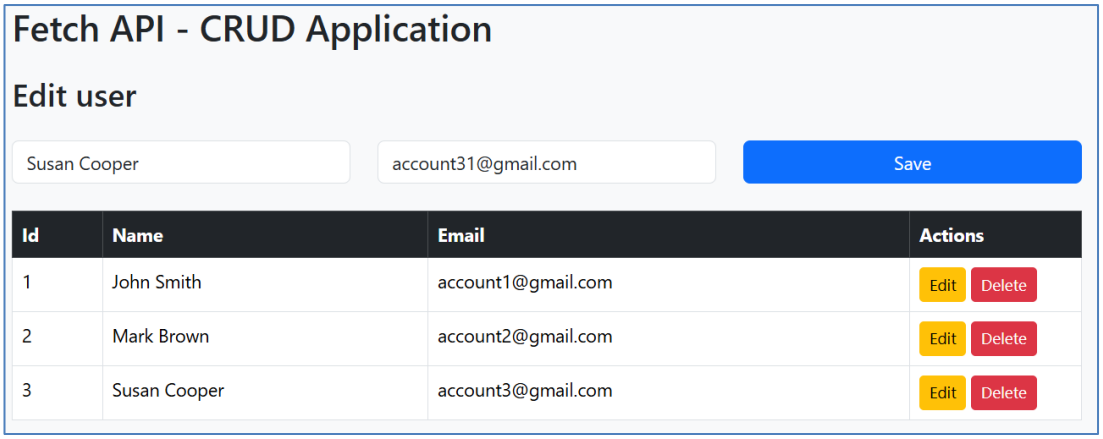


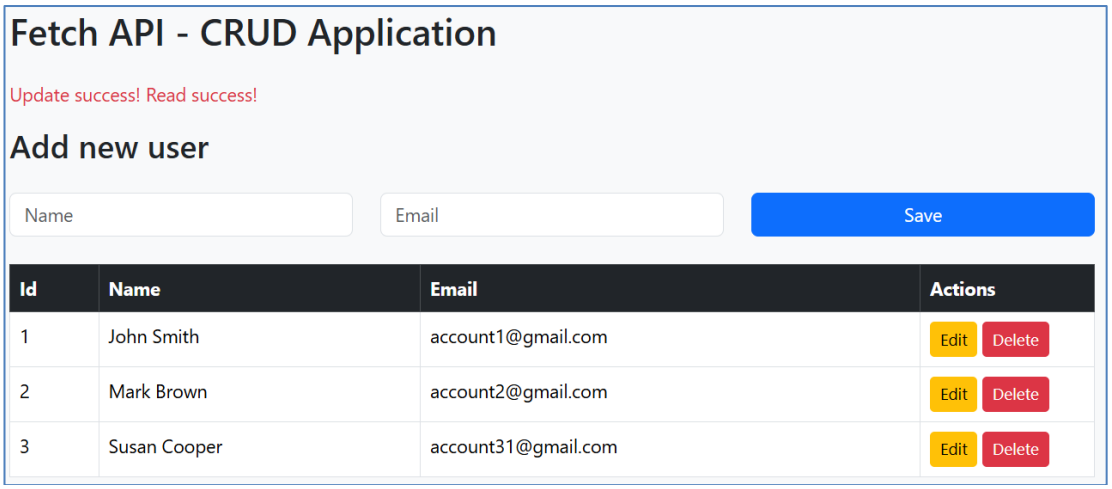
Create




Fetch API - CRUD Application
Create success! Read success!
Add new user
Name
Email
Save
Id
Name
Email
Actions
1
John Smith
account1@gmail.com
Edit
Delete
2
Mark Brown
account2@gmail.com
Edit
Delete
3
Susan Cooper
account3@gmail.com
Edit
Delete

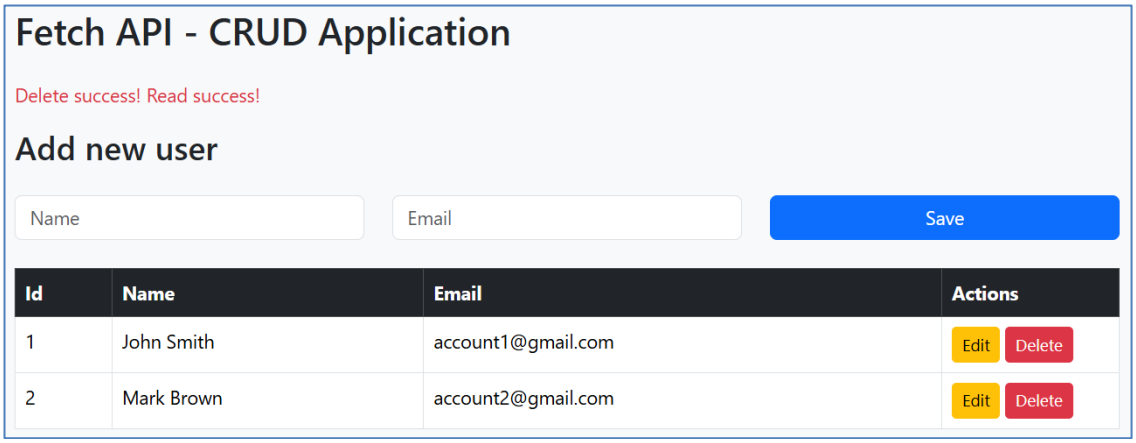
Edit


Fetch API - CRUD Application
Edit user
Susan Cooper
account31@gmail.com
Save
Id
Name
Email
Actions
1
John Smith
account1@gmail.com
Edit
Delete
2
Mark Brown
account2@gmail.com
Edit
Delete
3
Susan Cooper
account3@gmail.com
Edit
Delete


Fetch API - CRUD Application
Update success! Read success!
Add new user
Name
Email
Save
Id
Name
Email
Actions
1
John Smith
account1@gmail.com
Edit
Delete
2
Mark Brown
account2@gmail.com
Edit
Delete
3
Susan Cooper
account31@gmail.com
Edit
Delete

Delete:


Delete this user?
OK
Mégse



=> **Solutions.zip**

Exercise – 2 – POST method in all four cases – Only informative text

Here, in all four cases, we send the data using the POST method: We send the operation identifier in the message.

=> **Solutions.zip**

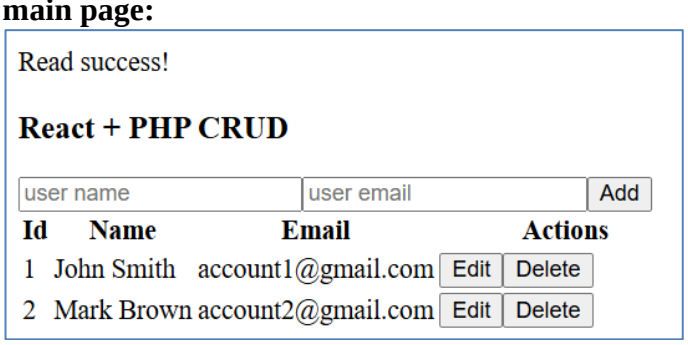
8 - React-Axios-CRUD – Similar to the JavaScript Fetch API exercise, only here we solve it with React and Axios - Design with AI

Design with AI:
ChatGPT: Create a CRUD application with React, PHP, MySQL, Axios. In PHP use the PDO class for the database. Use Bootstrap for formatting.
In PHP read method from $_SERVER['REQUEST_METHOD'];

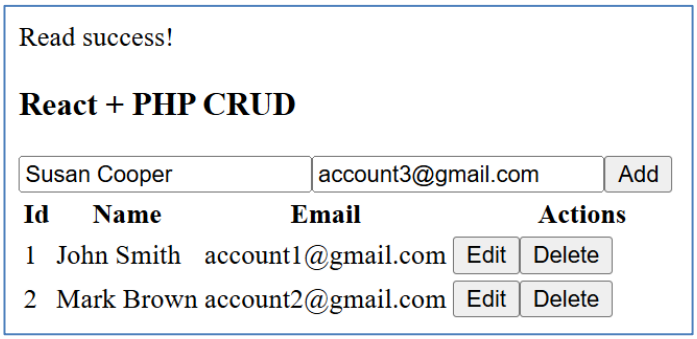
The resulting solution is transformed:

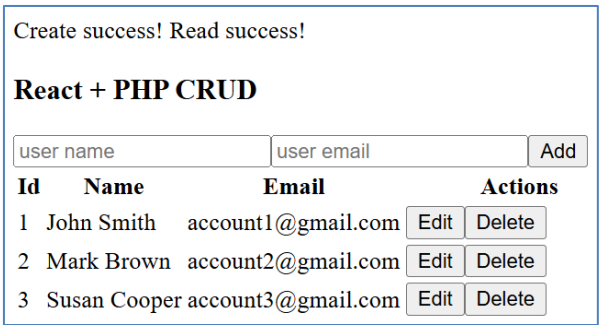
Exercise

main page:

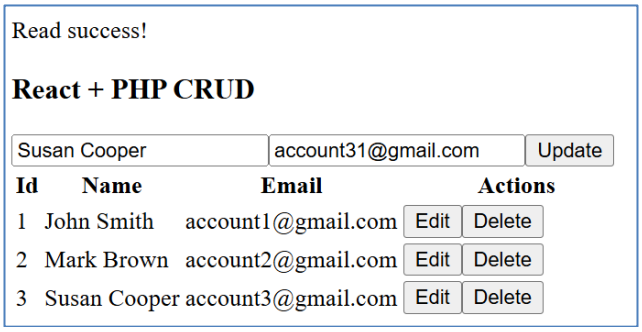


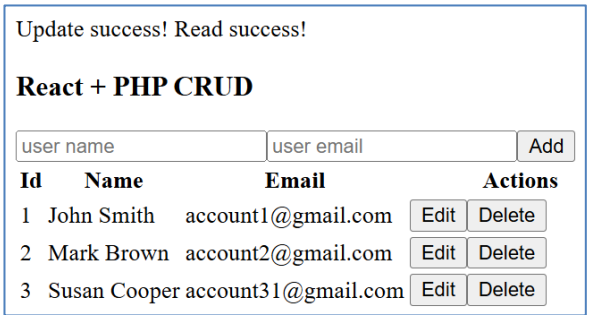
Create



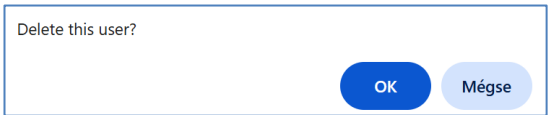


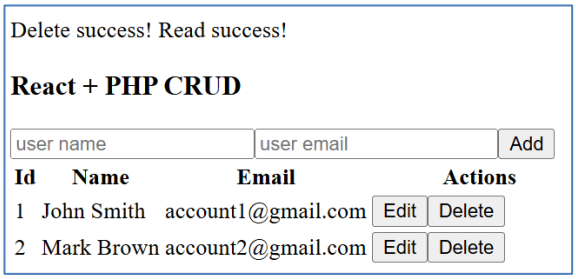
Edit





Delete:





1-Solution

Database (MySQL) – As in the previous exercise

```
CREATE DATABASE crud;
USE crud;
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
INSERT INTO users (name, email) VALUES
('John Smith', 'account1@gmail.com'),
('Mark Brown', 'account2@gmail.com');
```

Backend – As in the previous exercise

We implement it on localhost in the **c:\xampp\htdocs\exercise** folder.
The **db.php** and **api.php** are the same as the JavaScript Fetch API exercise:

Database Connection

c:\xampp\htdocs\exercise\db.php

```
<?php
$host = "localhost";
$db   = "crud";
$user = "root";
$pass = "";
try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=UTF8",$user,$pass,
    [PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION,PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC]);
} catch (PDOException $e) {
    die("Database connection failed");
}
```

PHP API
This single file handles **Create, Read, Update, Delete** via action.

c:\xampp\htdocs\exercise\api.php

```
<?php
header("Content-Type: application/json");
require "db.php";
$method = $_SERVER['REQUEST_METHOD'];
switch ($method) {
   case 'GET':
      try {
         $stmt = $pdo->query("SELECT * FROM users");
         $readData=$stmt->fetchAll();
         echo json_encode(['status' => 'Read success!', "readData"=>$readData]);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Read error!']);
      }
      break;
   case 'POST':
      try {
         $data = json_decode(file_get_contents("php://input"), true);
         $stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (?, ?)");
         $stmt->execute([$data['name'], $data['email']]);
         echo json_encode(['status' => 'Create success!']);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Create error!']);
      }
      break;
   case 'PUT':
      try {
         $data = json_decode(file_get_contents("php://input"), true);
         $stmt = $pdo->prepare("UPDATE users SET name=?, email=? WHERE id=?");
         $stmt->execute([$data['name'], $data['email'], $data['id']]);
         echo json_encode(['status' => 'Update success!']);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Update error!']);
      }
      break;
   case 'DELETE':
      try {
         $data = json_decode(file_get_contents("php://input"), true);
         $stmt = $pdo->prepare("DELETE FROM users WHERE id=?");
         $stmt->execute([$data['id']]);
         echo json_encode(['status' => 'Delete success!']);
      }
      catch(PDOException $e) {
        echo json_encode(['status' => 'Delete error!']);
      }
      break;
}
```

React Frontend

React development can be done in any folder, e.g. **c:\React** folder.
Unzip **Installed-React-*.*.*.zip** to **c:\React** folder

Install dependencies

npm install axios bootstrap
bootstrap is only needed for solution 2
If requested:
npm audit fix

Copy the solution from the **Solutions.zip** file**.**

npm run build

Copy the contents of the **dist** folder to the **c:\xampp\htdocs\exercise** folder.
Copy the **db.php** and **api.php** files to the **c:\xampp\htdocs\exercise** folder.

Start XAMPP

http://localhost/exercise
works well!

Since we are implementing it on localhost in the **c:\xampp\htdocs\exercise** folder, the **base** folder must be specified in the **vite.config.js** file:

vite.config.js
```
export default defineConfig({
  base: '/exercise/',
  plugins: [react()],
})
```

index.html
```
<!doctype html>
<html lang="en">
  <head>
    <title>react-app</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

/src/main.jsx
```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

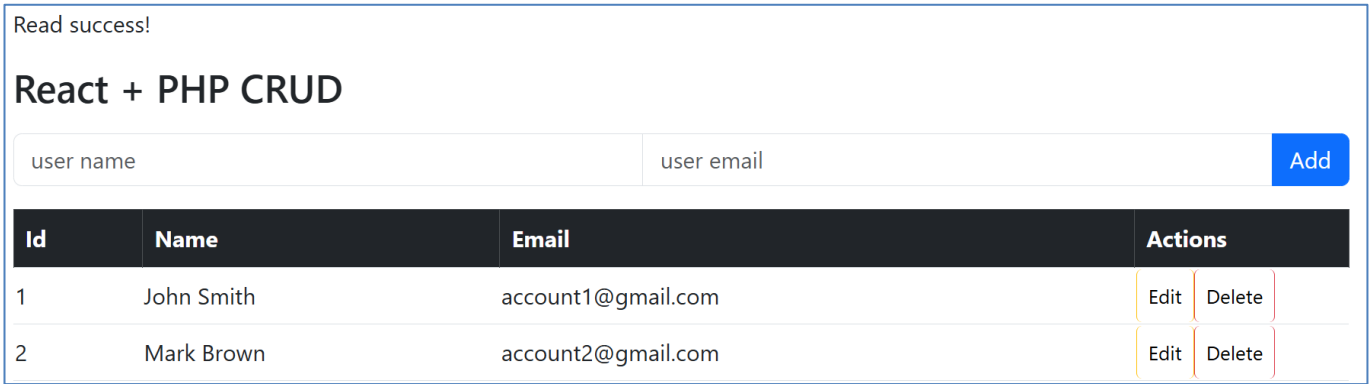
src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
// This is where we store the users read from the database:
  const [users, setUsers] = useState([]);
// This is where we store the message sent from api.php:
  const [message, setMessage] = useState("");
// This is where we store the value entered in the name input field:
  const [name, setName] = useState("");
// This is where we store the value entered in the email input field:
  const [email, setEmail] = useState("");
// This is where we store the user ID for the Edit(Update) operation:
  const [editId, setEditId] = useState(null);

// We learned: Don't change state while rendering! => otherwise it could be an infinite loop due to
//         re-rendering
//         We studied that we can avoid re-rendering by using useEffect if we use it like this:
//                 useEffect(() => {
//                 ……
//                 }, []);
// Runs on startup: read and display the users:
  useEffect(() => {
     fetchUsers();
  }, []);

// read and display the users. Called in several cases:
//         async: we learned about it in the JavaScript section
  const fetchUsers = async () => {
// We send a GET request to the api.php script using Axios:
    const res = await axios.get("api.php");
// we put the list of users returned from api.php into the users useState variable.
//         When rendering, it loads the user data into the page from this:
    setUsers(res.data.readData);
// adds the message from api.php to the message in message variable.
//         After adding a new user, message has this: Create success!
//         We solve this with the Updater function, which we have already encountered.
//         We saw that we use the updater function if we have already modified the value of the
//         useState variable in the event handler and now want to modify it again. Otherwise, the previous
//         would have no effect.
//         For example, in the Submit event handler, we modify the message useState variable:
//         setMessage(res.data.status);, and then we call the
//         fetchUsers(); function, in which we now modify it again.
//         Therefore, this would not be good here: setMessage(message+" "+res.data.status);
    setMessage(a => a+" "+res.data.status);
  };

// Event handler for clicking the Add/Update button next to the name and email input fields
//         There is this code on the button:
//         <button onClick={submit}> {editId ? "Update" : "Add"}</button>
  const submit = async () => {
```

```
  var res;
// editId: This is where we store the user ID for the Edit(Update) operation:
//        If editId is not an empty String, then Update(Put), otherwise Create(Post):
    if (editId) {
// In case of Update, we send a PUT request to the api.post script
//        We send the id, name, email values as parameters:
      res = await axios.put("api.php", { id: editId, name, email });
// delete the value of the edirId variable:
      setEditId(null);
    } else {
// When adding a new user (Create), we send a POST request to the api.post script
//        We send the name and email values as parameters:
      res = await axios.post("api.php", { name, email });
    }
// We put the message returned from api.php into the message variable. e.g. Create success!
    setMessage(res.data.status);
// Delete the values of the name and email variables:
    setName("");
    setEmail("");
// read and display the users.
    fetchUsers();
  };

// Event handler for clicking the Edit button next to user on the page
//                  <button onClick={() => editUser(user)}>Edit</button>
//        The selected user's data is put into the useState variables.
//        The id is stored in the editId field.
//        The name and email are displayed in the input fields after rendering.
  const editUser = (user) => {
    setEditId(user.id);
    setName(user.name);
    setEmail(user.email);
  };

// Event handler for clicking the Delete button next to user on the page
  const deleteUser = async (id) => {
// Opens a Confirm window to confirm the deletion:
    if (!confirm("Delete this user?")) return;
// The Axios delete method should have the parameter in this format:
//        const res = await axios.delete("api.php", {data:{id}});
//        This would not be good: const res = await axios.delete("api.php", {id});
    const res = await axios.delete("api.php", {data:{id}});
    setMessage(res.data.status);
    fetchUsers();
  };

// Displays the page when rendering:
  return (
    <div>
// Display the message useState variable:
      <p>{message}</p>
      <h3>React + PHP CRUD</h3>
      <div>
```

```
// The name input field:
//        When modified, it saves its current value in the name useState variable.
       <input value={name} onChange={(e) => setName(e.target.value)} placeholder="user name"/>
       <input value={email} onChange={(e) => setEmail(e.target.value)} placeholder="user email" />
// Button that appears next to the input fields
//        If editId is not an empty string, it is labeled "Update", otherwise "Add"
//        The event handler for clicking the button is the submit function
       <button onClick={submit}> {editId ? "Update" : "Add"}</button>
     </div>
// Displays the users' data in a table:
     <table>
       <thead>
         <tr>
           <th>Id</th>
           <th>Name</th>
           <th>Email</th>
           <th width="150">Actions</th>
         </tr>
       </thead>
// Loop through the users:
       {users.map((user) => (
         <>
           <tr>
             <td>{user.id}</td>
             <td>{user.name}</td>
             <td>{user.email}</td>
             <td>
               <button onClick={() => editUser(user)}>Edit</button>
               <button onClick={() => deleteUser(user.id)}>Delete</button>
             </td>
           </tr>
         </>
       ))}
     </table>
   </div>
  );
}
export default App;
```

2-Formatting with Bootstrap – 10 mins



=> **Solutions.zip**

9 – Object oriented JavaScript

We have already met object-oriented JavaScript, since **JavaScript is an object-oriented (OOP) programming language**. **All objects in JavaScript** (except primitives).
Even in the document.write(…) instruction used in the first exercises, the **document is an object** whose method is write(…).
So the **novelty** will not be object orientation, but e.g. class creation, constructor, inheritance, class, …

Javascipt is an object-oriented programming language from the beginning, but the **class declaration** (only appeared in ECMAScript 6 (2015). Before that, inheritance was implemented using **prototypes** and not classes.

1-2 exercises

Solve tasks first by using prototypes and then by using the language elements introduced in ECMAScript 6 to define classes.

1st exercise
Create an **Image** class for handling image objects, which:

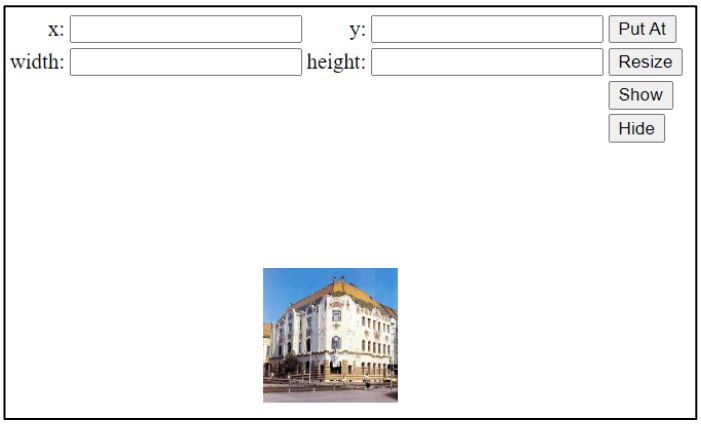
a) Its constructor receives the following parameters:
- **src**: the url of the source file of the image
- **x, y**: the position of the upper left corner of the image within the browser window,
- **width, height**: dimensions of the image.

b) Its constructor initializes the **img** attribute of the object based on the received parameters (to create it, use: **document.createElement("img");** to place it in the **DOM**, use: **document.body.appendChild(this.image)** and set the appropriate style properties.

c) Its methods are as follows:
- **show**: displays the image in the browser,
- **hide**: hides the image,
- **putAt(x,y):** moves the upper left corner of the image to the position received as a parameter,
- **resize(width,height)**: resizes the image to the size given as a parameter.

d) Test the developed class: create an html file for it.

The task includes the file **example.jpg**, which you can find in the solutions.



2nd exercise

As in the previous task, there is also a subtitle under the image, which must be moved together with the image. We use the previous **Image** class and derive the **SubtitledImage** class from it.

A, Soluion – with prototypes – Only informative text

Can be found in **Solutions.zip**

B, Solution – With classes

B/1. solution – without subtitle
We create 2 files: **obj.js, obj.html**

obj.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Objects in JavaScript</title>
   <script type="text/javascript" src="obj.js">
   </script>
</head>
<body>
   <script type="text/javascript">
// Calling with the new operator returns an object
//         places the given image in the given location with the given size
// Create the Image object constructor in obj.js
//         200, 200: upper left corner x,y coordinate
//         100, 100: width, height
      var myimage = new Image("example.jpg", 200, 200, 100, 100);
   </script>
// puts elements in a table
   <table>
      <tr>
// text boxes for entering data
      <td align="right">x:</td><td><input type="text" id="x" name="x"></td>
      <td align="right">y:</td><td><input type="text" id="y" name="y"></td>
// To press the Put At button, call the putAt method of the myimage object.
// Places the image in the location specified in the two text boxes
      <td><button onclick="myimage.putAt(x.value,y.value)">Put At</button></td>
      </tr>
      <tr>
      <td align="right">width:</td><td><input type="text" id="width" name="width"></td>
      <td align="right">height:</td><td><input type="text" id="height" name="height"></td>
// Resize the image to the value specified in the two text boxes
      <td><button onclick="myimage.resize(width.value,height.value)">Resize</button></td>
      </tr>
      <tr>
      <td colspan="4"></td>
// show the image
      <td><button onclick="myimage.show()">Show</button></td>
      </tr>
      <tr>
      <td colspan="4"></td>
// Hides the image
```

```
      <td><button onclick="myimage.hide()">Hide</button></td>
      </tr>
   </table>
</body>
</html>
```

obj.js

```
class Image {
// Constructors can be defined with the optional constructor attribute.
// We do not need to specify it if the name of the creating function is the same as that of the class.
// This will be the so-called default constructor
// Creates and places the image object on the page when instantiated:
   constructor(src, x, y, width, height) {
// this.img = … We define a (global) variable (property) img for the class
      this.img = document.createElement("img");
      this.img.src = src;
      this.img.style.position = "absolute";
      this.img.style.left = x+"px";
      this.img.style.top = y+"px";
      this.img.width = width;
      this.img.height = height;
      document.body.appendChild(this.img);
   }
   show() {
      this.img.style.visibility = "visible";
   }
   hide() {
      this.img.style.visibility = "hidden";
   }
   putAt(x, y) {
      this.img.style.left = x+"px";
      this.img.style.top = y+"px";
   }
   resize(width, height) {
      this.img.width = width;
      this.img.height = height;
   }
}
```

B/2. solution – With subtitle

objsub.html - is the same as at A/2 solution

```
<!DOCTYPE html>
<html>
<head>
   <title>Objects in JavaScript</title>
   <script type="text/javascript" src="objsub.js">
   </script>
</head>
<body>
   <script type="text/javascript">
      var myimage = new SubtitledImage("example.jpg", 200, 200, 100, 100, "With subtitle");
   </script>
```

```
    <table>
      <tr>
      <td align="right">x:</td><td><input type="text" id="x" name="x"></td>
      <td align="right">y:</td><td><input type="text" id="y" name="y"></td>
      <td><button onclick="myimage.putAt(x.value,y.value)">Put At</button></td>
      </tr>
      <tr>
      <td align="right">width:</td><td><input type="text" id="width" name="width"></td>
      <td align="right">height:</td><td><input type="text" id="height" name="height"></td>
      <td><button onclick="myimage.resize(width.value,height.value)">Resize</button></td>
      </tr>
      <tr>
      <td colspan="4"></td>
      <td><button onclick="myimage.show()">Show</button></td>
      </tr>
      <tr>
      <td colspan="4"></td>
      <td><button onclick="myimage.hide()">Hide</button></td>
      </tr>
    </table>
</body>
</html>
```

objsub.js

```
// this class is the same as in obj.js
class Image {
    constructor(src, x, y, width, height) {
        this.img = document.createElement("img");
        this.img.src = src;
        this.img.style.position = "absolute";
        this.img.style.left = x+"px";
        this.img.style.top = y+"px";
        this.img.width = width;
        this.img.height = height;
        document.body.appendChild(this.img);
    }

    show() {
        this.img.style.visibility = "visible";
    }

    hide() {
        this.img.style.visibility = "hidden";
    }

    putAt(x, y) {
        this.img.style.left = x+"px";
        this.img.style.top = y+"px";
    }

    resize(width, height) {
        this.img.width = width;
```

```
        this.img.height = height;
    }
}

class SubtitledImage extends Image {
    constructor(src, x, y, width, height, subtitle) {
// calls the default constructor of the parent class (Image):
// Creates and places the image object on the page on instantiation
        super(src, x, y, width, height);
        this.subtitle = document.createElement("div");
        this.subtitle.innerHTML = subtitle;
        this.subtitle.style.position = "absolute";
        this.subtitle.style.left = x+"px";
        this.subtitle.style.top = (y+height)+"px";
        document.body.appendChild(this.subtitle);
    }

    show() {
// Calls the show() method of the parent class (Image).
        super.show();
        this.subtitle.style.visibility = "visible";
    }

    hide() {
        super.hide();
        this.subtitle.style.visibility = "hidden";
    }

    putAt(x, y) {
        super.putAt(x, y);
        this.subtitle.style.left = x+"px";
        this.subtitle.style.top = (parseInt(y)+parseInt(this.img.height))+"px";
    }

    resize(width, height) {
        super.resize(width, height);
        this.subtitle.style.top = (parseInt(this.img.style.top)+parseInt(height))+"px";
    }
}
```

3rd exercise – Only informative text

Write a JavaScript source file called exercise3.js that defines the following classes:

1. Define the **UniversityEmployee** class, whose member variables are **name**, **address** and **salary**; the **salaryModify** method adds the value received as a parameter to the value of the salary attribute.
2. Define the **Teacher** class, which is an extension of the **UniversityEmployee** class, its additional data members are **department** and the initially empty **courses** array, its methods are the **course** that takes the subject received as a parameter, and **numberCourses**, which returns the number of subjects.

The result of executing the specified test.html can be seen in the image:

Initial table

Name	Address	Salary	Department	Courses
John Smith	Budapest	450000	---	---
Kate Brown	Kecskemét	500000	Informatics	0:
Tom Green	Szeged	475000	Mechanics	0:

Table after modification

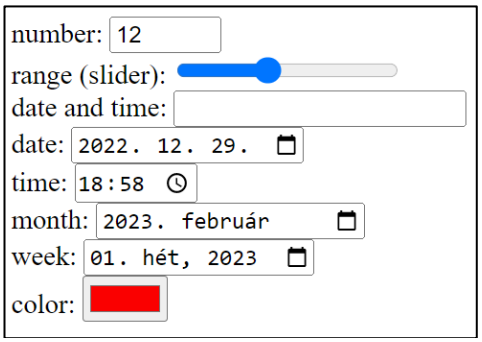
Name	Address	Salary	Department	Courses
John Smith	Budapest	500000	---	---
Kate Brown	Kecskemét	500000	Informatics	2: Programming 1,Programming 2
Tom Green	Szeged	500000	Mechanics	0:

=> Solutions.zip

10 - New features of HTML5 and ChartJS

We studied in seminar: <header>, <nav>, <article>, <footer>, <aside> tags

New HTML5 input elements



It isn't in **Solutions.zip**

……

```
<body>
        number: <input type="number" min="10" max="20" step="2"><br>
        range (slider): <input type="range" min="0" max="10" step="2" value="6"><br>
        date and time: <input type="datetime"><br>
        date: <input type="date"><br>
        time: <input type="time"><br>
        month: <input type="month"><br>
        week: <input type="week"><br>
        color: <input type="color">
</body>
```

……

Web Storage – better than cookies

Disadvantages and Limitations of Cookies

- They go with every HTML message, slowing down communication
- Cookies go unencrypted in messages
- The size of the cookie is limited (4 kB)

Two new mechanisms in HTML5

- Session storage
- Local storage

Their use is justified in different situations
Modern browsers all support

Session Storage

The user performs a simple process in a browser window (there can be multiple processes in different windows)

sessionStorage variable:

- stores data in a session
- once the user closes the window, its contents are lost

The next example counts the number of times the user has refreshed the page in the current session. Closing and reopening the browser tab restarts the page count from 1.

01-sessionstorage/index.html

```
<!DOCTYPE HTML>
<html>
<body>
	<script type="text/javascript">
		if( sessionStorage.hits )
		  sessionStorage.hits = Number(sessionStorage.hits) +1;
		else
		  sessionStorage.hits = 1;
		document.write("Total Hits :" + sessionStorage.hits );
	</script>
	<p>Refresh the page to increase number of hits.</p>
	<p>Close the window and open it again and check the result.</p>
</body>
</html>
```

Local Storage

localStorage variable:

- the stored data is shared by multiple open windows/browser tabs
- its value remains even after the window is closed
- no size limit (we can even store user mailboxes or documents)

Sensitive data should not be stored for long periods

- delete a variable: localStorage.remove('hits')
- delete all variables: localStorage.clear()

Like the previous exercise, but closing and reopening the browser tab will continue the calculation from where it left off (the previous state is preserved)

02-localstorage/index.html

```
<!DOCTYPE HTML>
<html>
<body>
	<script type="text/javascript">
		if( localStorage.hits )
			localStorage.hits = Number(localStorage.hits) +1;
		else
			localStorage.hits = 1;
		document.write("Total Hits :" + localStorage.hits );
	</script>
	<p>Refresh the page to increase number of hits.</p>
	<p>Close the window and open it again and check the result.</p>
</body>
</html>
```

Web Workers – Running JavaScript code in the background - Parallel programming with JavaScript

JavaScript is designed to run single-threaded
Execution that takes longer (e.g. long cycles) will block the browser

03-web_workers/index-without-web-worker.html

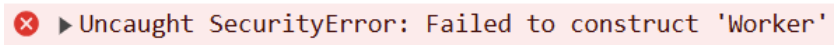
```
<!DOCTYPE HTML>
<html>
<head>
<title>Big for loop</title>
 <script>
    function bigLoop(){
      for (var i = 0; i <= 10000000000; i += 1)
        var j = i;
      alert("Completed " + j + "iterations" );
    }
    function sayHello(){
      alert("Hello sir...." );
    }
 </script>
</head>
<body>
  <input type="button" onclick="bigLoop();" value="Big Loop">
  <input type="button" onclick="sayHello();" value="Say Hello">
</body>
</html>
```

with-Web-worker

A web server is also required.

Due to security concerns, access to local files from browsers is restricted. Web browsers (and JavaScript) can only access local files **with user permission**.

right click on page / Inspect / Console



You have already learned server-side programming in Seminar. Both solutions are good:
A, with PHP Development Server:
in the folder where the html file is, at the command line:
\xampp\php\php -S localhost:80
http://localhost/index-with-web-worker.html
B, with XAMPP

03-web_workers / bigLoop.js

```
for (var i = 0; i <= 1000000000; i += 1)
  var j = i;
postMessage(j);        // post a message back to the HTML page.
```

Since workers run in a separate thread from the one that created them, communication needs to happen via postMessage. The **postMessage()** method of the Worker interface sends a message to the worker's inner scope. This accepts a single parameter, which is the data to send to the worker.

03-web_workers / index-with-web-worker.html

```
<!DOCTYPE HTML>
<html>
	<head>
		<meta charset="utf-8">
		<title>Web Worker</title>
		<script>
			function bigLoop(){
				if (typeof(Worker) !== "undefined") {
					var worker = new Worker('bigLoop.js');
					worker.onmessage = function (event) {
						alert("Completed " + event.data + "iterations" );
					};
				} else {
					alert("Sorry, your browser does not support Web Workers..." );
				}
			}
			function sayHello(){
				alert("Hello...." );
			}
		</script>
	</head>
	<body>
		<input type="button" onclick="bigLoop();" value="Big Loop">
		<input type="button" onclick="sayHello();" value="Say Hello">
	</body>
</html>
```

The Web Worker can be stopped from the script that started it using the worker.terminate() method.

Server Sent Events - SSE – It improves the limitation of AJAX: always the client initiates there

We will learn Ajax later.

The limitation of AJAX: always the client initiates
In the case of multi-player events (chat, games), the server cannot initiate the sending
Solutions with traditional technology:

- **polling technique** (JS timer and AJAX): retrieval per time unit; displaying “newness”
  Disadvantage: a lot of unnecessary server load; opening-closing a new connection every time
- **long polling**: when the request arrives: the server only sends a response if there is new data, otherwise it waits to send back

Real solutions: WebSocket and SSE

WebSocket:

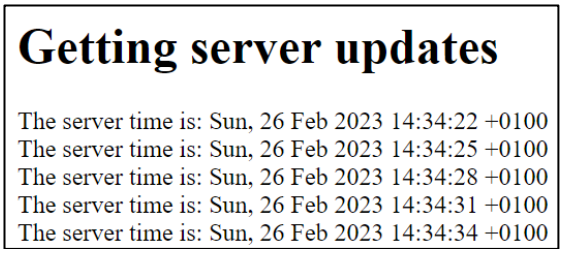
- does not use HTTP protocol

SSE:

- uses HTTP protocol
- completely handled by the browser
- easy to program on the server side
- does not generate unnecessary load during creation-destruction

You have already learned server-side programming in Seminar. Both solutions are good:
A, with PHP Development Server:
in the folder where the html file is, at the command line:
\xampp\php\php -S localhost:80
http://localhost/index-with-web-worker.html
B, with XAMPP

Exercise-1



……

demo_sse.php

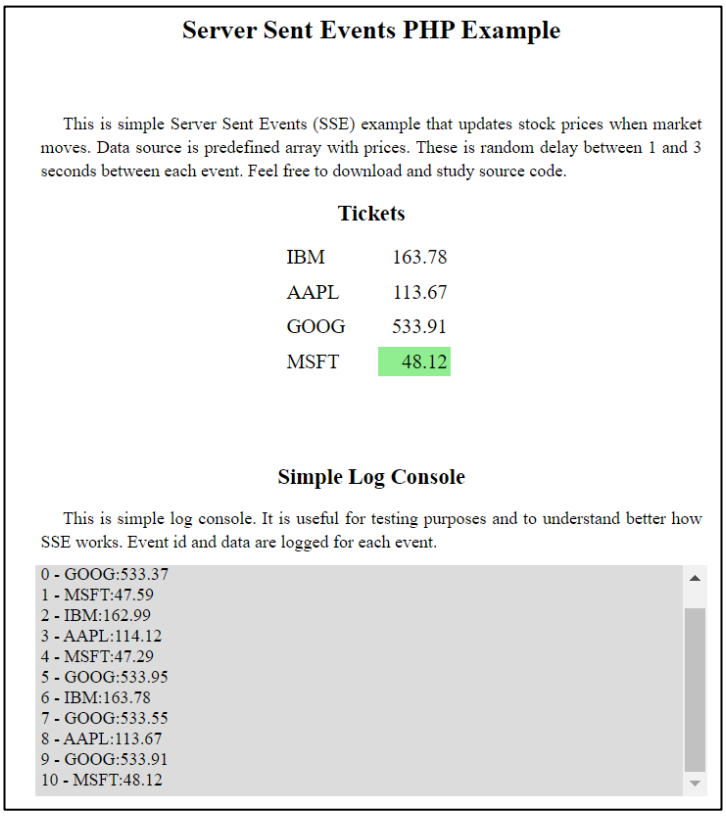
```
<?php
	header('Content-Type: text/event-stream');
	header('Cache-Control: no-cache');
	$time = date('r');
	echo "data: The server time is: {$time}\n\n";
	flush();
?>
```

index.html

```
<!DOCTYPE html>
<html>
<body>
	<h1>Getting server updates</h1>
	<div id="result"></div>
	<script>
		if(typeof(EventSource) !== "undefined") {
			var source = new EventSource("demo_sse.php");
			source.onmessage = function(event) {
				document.getElementById("result").innerHTML += event.data + "<br>";
			};
		} else
			document.getElementById("result").innerHTML = "Sorry, your browser does not
support server-sent events...";
	</script>
</body>
</html>
```

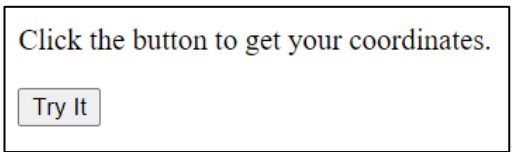
Exercise-2 – We just test it, we don't analyze the code

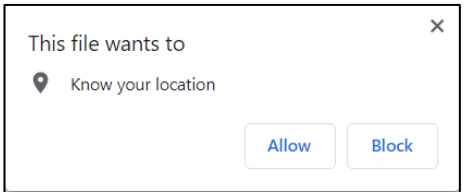
The page updates the stock price when it changes:

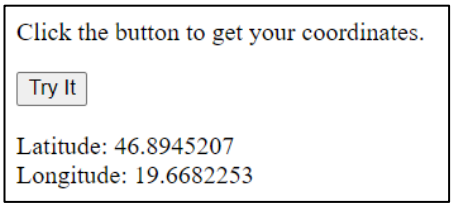


Geolocation API

- Querying the user's geographic location
- Personal data, can only be retrieved with the user's consent





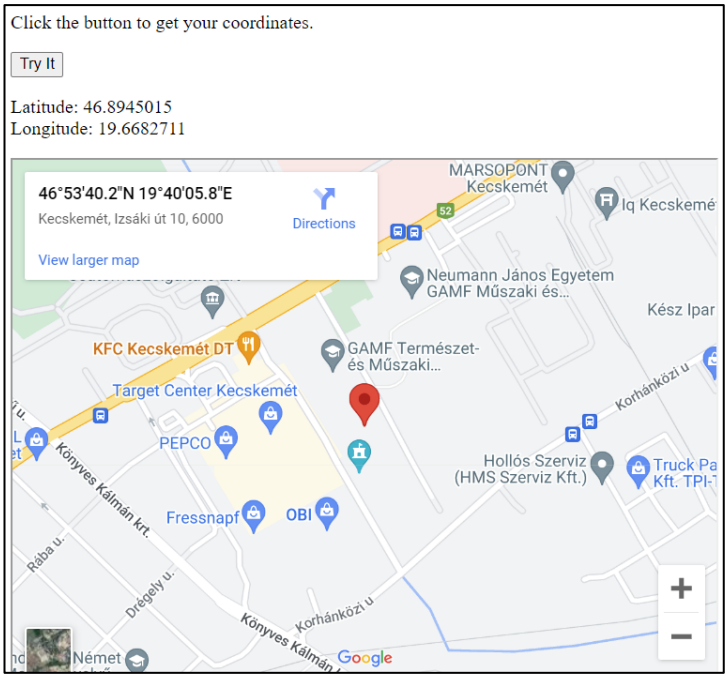


05-geolocation / index.html

```
<!DOCTYPE html>
<html>
<body onload="myFunction()">
        <p>Click the button to get your coordinates.</p>
        <button onclick="getLocation()">Try It</button>
        <p id="demo"></p>
```

```
	<script>
		var x = document.getElementById("demo");
		function getLocation()  {
			if (navigator.geolocation)
				navigator.geolocation.getCurrentPosition(showPosition);
			else
				x.innerHTML = "Geolocation is not supported by this browser.";
		}
		function showPosition(position)  {
			x.innerHTML = "Latitude: " + position.coords.latitude + "<br>Longitude: " +
position.coords.longitude;
		}
	</script>
</body>
</html>
```

Positions inserted into Google maps – Only informative text – we try it, but we don't analyze the code



05-geolocation / index2.html

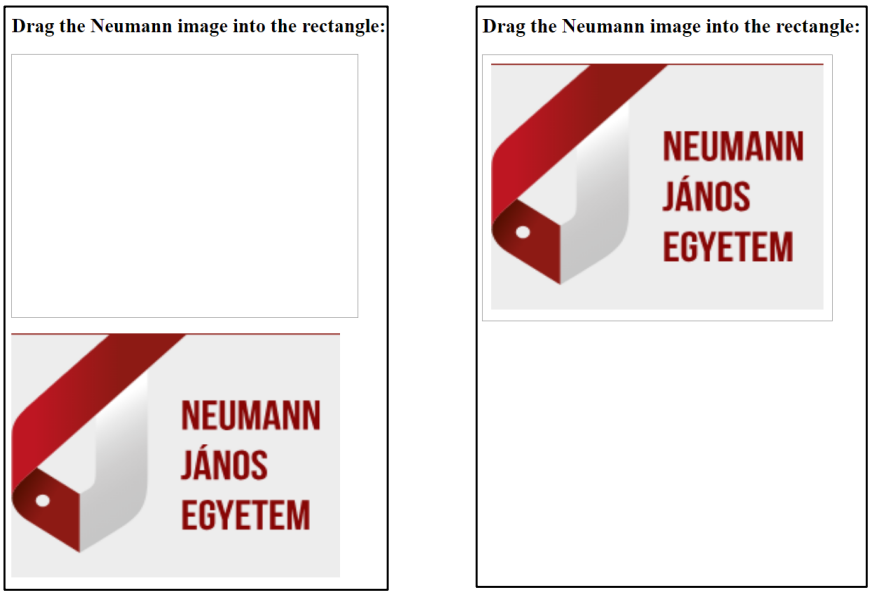
```
<!DOCTYPE html>
<html>
<body onload="myFunction()">
	<p>Click the button to get your coordinates.</p>
	<button onclick="getLocation()">Try It</button>
	<p id="demo"></p>
	<script>
		var x = document.getElementById("demo");
		function getLocation()  {
			if (navigator.geolocation)
				navigator.geolocation.getCurrentPosition(showPosition);
			else
				x.innerHTML = "Geolocation is not supported by this browser.";
		}
		function showPosition(position)  {
```

```
                x.innerHTML = "Latitude: " + position.coords.latitude + "<br>Longitude: " +
position.coords.longitude;
                var newContent = '<iframe src = "https://maps.google.com/maps?q=' +
position.coords.latitude + ',' + position.coords.longitude + '&hl=es;z=14&amp;output=embed"
width="600" height="450"></iframe>';
                var contentHolder = document.getElementById('content-holder');
                contentHolder.innerHTML = newContent;
            }
        </script>
        <p id="content-holder">Google maps: Waiting for GPS coordinates ...</p>
</body>
</html>
```

Additional methods:
watchPosition() – returns the position continuously (e.g. for a program running on moving vehicles)
clearWatch() – stops watchPosition()

Drag and drop API

Exercise-1



- img
  - draggable="true": make the element draggable
  - ondragstart="drag(event)": Call the drag(event) function when dragging
- div
  - ondragover: allows other elements to be dropped here
  - ondrop: specifies what happens when dropped

index.html

```
<!DOCTYPE HTML>
<html>
<head>
        <style>
            #div1 {width:400px;height:300px;padding:10px;border:1px solid #aaaaaa;}
        </style>
```

```
	<script>
// A drop engedélyezéséhez megakadályozzuk az elem alapértelmezett kezelését.
//	Enélkül nem működik a drop:
//	https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/drop_event
		function allowDrop(ev) {
			ev.preventDefault();
		}
		function drag(ev) {
// The dataTransfer.setData() method sets the data type and value of the transferred data.
//	data type of ev.target.id: "text"
//	ev.target.id: ID of the dragged item
			ev.dataTransfer.setData("text", ev.target.id);
		}
		function drop(ev) {
			ev.preventDefault();
// dataTransfer.getData(): gets the transferred data
			var data = ev.dataTransfer.getData("text");
// appending the transferred element to the target element in the DOM:
			ev.target.appendChild(document.getElementById(data));
		}
	</script>
</head>
<body>
	<h2>Drag the Neumann image into the rectangle:</h2>
// ondragover: we allow other elements to be dropped here
//	By default, data/elements cannot be dropped into other elements.
// ondrop: we specify what happens when dropped
	<div id="div1" ondrop="drop(event)" ondragover="allowDrop(event)"></div>
	<br>
// draggable="true": Make the element draggable
// ondragstart="drag(event)": when dragging, we call the drag(event) function
	<img id="drag1" src="nje.png" draggable="true" ondragstart="drag(event)" width="400"
height="300">
</body>
</html>
```

Exercise-2 - Back and forth – Only informative text – we try it, but we don't analyze the code

code: back-and-forth.html

Creating two-dimensional graphics and animations with JavaScript

```
<canvas id="mycanvas" width="100" height="100"></canvas>
```

Checking browser support, extracting drawing environment

```
var canvas =  document.getElementById("mycanvas");
if (canvas.getContext){
  var ctx = canvas.getContext('2d');
} else { ... // unsupported canvas }
```

The <canvas> element is just a container for the graphics. We need to use JavaScript to actually draw the graphics.

Draw the Canvas frame:

01-canvas.html

```
<!DOCTYPE HTML>
<html>
  <head>
    <style>
      #mycanvas{border:1px solid red;}
    </style>
  </head>
  <body>
     <canvas id="mycanvas" width="100" height="100"></canvas>
  </body>
</html>
```

Draw rectangles and squares

02-rectangles.html

```
<!DOCTYPE HTML>
<html>
  <head>
    <style>
      #test {width: 100px; height:100px; margin: 0px auto;}
    </style>
    <script type="text/javascript">
      function drawShape(){
        var canvas = document.getElementById('mycanvas'); // Get the canvas element using the DOM
        if (canvas.getContext){ // Make sure we don't execute when canvas isn't supported
          var ctx = canvas.getContext('2d'); // use getContext to use the canvas for drawing
// The fillRect() method draws a "filled" rectangle. The default color of the fill is black.
          ctx.fillRect(25,25,100,100); // Draw shapes
// The clearRect() method clears the specified pixels within a given rectangle.
          ctx.clearRect(45,45,60,60);
// The strokeRect() method draws a rectangle (no fill). The default color of the stroke is black.
          ctx.strokeRect(50,50,50,50);
        }
        else
          alert('Your browser doesn\'t support Canvas.');
      }
    </script>
  </head>
```

```
  <body id="test" onload="drawShape();">
    <canvas id="mycanvas"></canvas>
  </body>
</html>
```

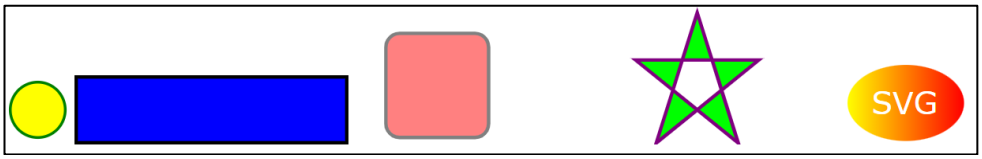
További Canvas példák – Csak olvasmány

A **Megoldások.zip** fájlban talál további példákat.

SVG - Graphics with HTML – without JavaScript

SVG: Scalable Vector Graphics. SVG defines vector-based graphics in XML format.

Let's draw some shapes with SVG:



svg.html

```
<!DOCTYPE html>
<html>
<body>
        <svg width="100" height="100">
          <circle cx="50" cy="50" r="40" stroke="green" stroke-width="4" fill="yellow" />
        </svg>
        <svg width="400" height="100">
          <rect width="400" height="100" style="fill:rgb(0,0,255);stroke-width:10;stroke:rgb(0,0,0)" />
        </svg>
        <svg width="400" height="180">
          <rect x="50" y="20" rx="20" ry="20" width="150" height="150"
style="fill:red;stroke:black;stroke-width:5;opacity:0.5" />
        </svg>
        <svg width="300" height="200">
          <polygon points="100,10 40,198 190,78 10,78 160,198" style="fill:lime;stroke:purple;stroke-
width:5;fill-rule:evenodd;" />
        </svg>
        <svg height="130" width="500">
                <defs>
                 <linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="0%">
                        <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1" />
                        <stop offset="100%" style="stop-color:rgb(255,0,0);stop-opacity:1" />
                 </linearGradient>
                </defs>
                <ellipse cx="100" cy="70" rx="85" ry="55" fill="url(#grad1)" />
                <text fill="#ffffff" font-size="45" font-family="Verdana" x="50" y="86">
                 SVG
                </text>
                Sorry, your browser does not support inline SVG.
        </svg>
```

</body>
</html>

SVG vs Canvas

- **SVG**: language for describing 2D graphics in XML
  SVG is XML-based, all its elements are available in the SVG DOM; JavaScript event handlers can be assigned to the elements
  In SVG, the drawn shape is an object, the appearance of which is immediately modified by the browser if its attributes are changed
  Supports event handlers
- **Canvas**: draws 2D graphics using JS
  Canvas: display per pixel; the browser "does not know" the elements; for modification, the objects (and what they cover) must be redrawn
  Does not support event handlers

SVG	Canvas
SVG uses geometric shapes to render graphics	Canvas uses pixels
Vector based (composed of shapes)	Raster based (composed of pixel)
SVG has better scalability. So it can be printed with high quality at any resolution.	Canvas has poor scalability. Hence it is not suitable for printing on higher resolution.
SVG gives better performance with smaller number of objects or larger surface.	Canvas gives better performance with smaller surface or larger number of objects.
SVG can be modified through script and CSS.	Canvas can be modified through script only.
Multiple graphical elements, which become the part of the page's DOM tree.	Single element similar to <img> in behavior. Canvas diagram can be saved to PNG or JPG format.

ChartJS – Drawing graphs with Canvas and JavaScript

https://www.chartjs.org/
https://www.chartjs.org/docs/latest/samples/information.html
Many types of graphs can be drawn.

Készítsük el a következő 2 adatsorból álló vonaldiagramot:

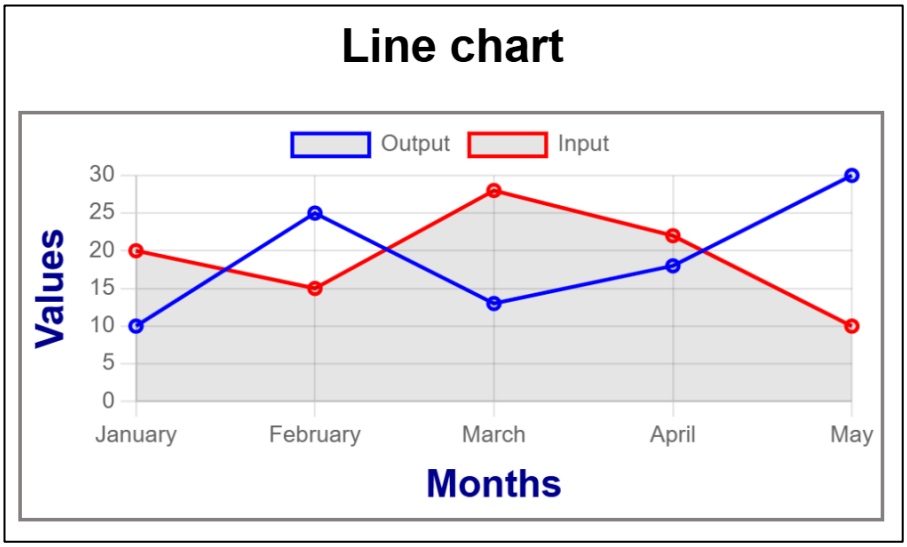


chart1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Chart.js Line Chart</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
        body {
            font-family: 'Arial', sans-serif;
            margin: 20px;
            text-align: center;
        }
        h1 {
            color: green;
        }
        canvas {
            border: 2px solid #858080;
        }
    </style>
</head>
<body>
    <h2>Line chart</h2>
    <canvas id="myLineChart" width="380" height="180"></canvas>
    <script>
        // data for showing the line chart
        let labels = ['January', 'February', 'March', 'April', 'May'];
        let dataset1Data = [10, 25, 13, 18, 30];
        let dataset2Data = [20, 15, 28, 22, 10];

        // Creating line chart
        let ctx = document.getElementById('myLineChart').getContext('2d');
        let myLineChart = new Chart(ctx, {
            type: 'line',
            data: {
                labels: labels,
```

```
        datasets: [
            {
                label: 'Output',
                data: dataset1Data,
                borderColor: 'blue',
                borderWidth: 2,
                fill: false,
            },
            {
                label: 'Input',
                data: dataset2Data,
                borderColor: 'red',
                borderWidth: 2,
                fill: true,
            },
        ]
    },
    options: {
        responsive: true,
        scales: {
            x: {
                title: {
                    display: true,
                    text: 'Months',
                    font: {
                        padding: 4,
                        size: 20,
                        weight: 'bold',
                        family: 'Arial'
                    },
                    color: 'darkblue'
                }
            },
            y: {
                title: {
                    display: true,
                    text: 'Values',
                    font: {
                        size: 20,
                        weight: 'bold',
                        family: 'Arial'
                    },
                    color: 'darkblue'
                },
                beginAtZero: true,
                scaleLabel: {
                    display: true,
                    labelString: 'Values',
                }
            }
        }
    }
});
```

```
    </script>
</body>
</html>
```

Chart2 – Only informative text

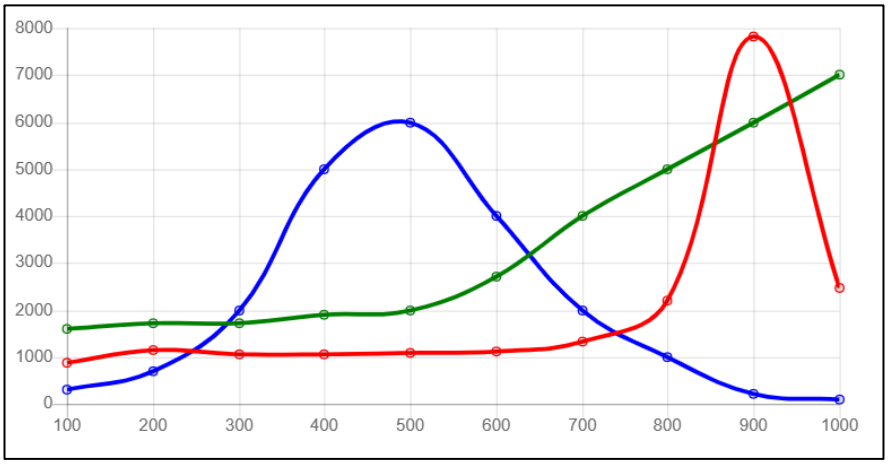


chart2.html

```
<!DOCTYPE html>
<html>
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/2.5.0/Chart.min.js"></script>
<body>
<canvas id="myChart" style="width:100%;max-width:600px"></canvas>

<script>
const xValues = [100,200,300,400,500,600,700,800,900,1000];

new Chart("myChart", {
  type: "line",
  data: {
    labels: xValues,
    datasets: [{
      data: [860,1140,1060,1060,1070,1110,1330,2210,7830,2478],
      borderColor: "red",
      fill: false
    }, {
      data: [1600,1700,1700,1900,2000,2700,4000,5000,6000,7000],
      borderColor: "green",
      fill: false
    }, {
      data: [300,700,2000,5000,6000,4000,2000,1000,200,100],
      borderColor: "blue",
      fill: false
    }]
  },
  options: {
    legend: {display: false}
  }
});
```

```
</script>
```